

UNIwersytet Warmiński-Mazurski w Olsztynie
Wydział Matematyki i Informatyki

Kierunek: Informatyka

Jakub Michalski

**Kafelkowa obrona – gra z proceduralnie
generowanym światem w OpenGL**

Praca inżynierska wykonana
w Katedrze Metod Matematycznych Informatyki
pod kierunkiem
dr inż. Bartosza Nowaka

Olsztyn, 2026 rok

UNIVERSITY OF WARMIA AND MAZURY IN OLSZTYN
FACULTY OF MATHEMATICS AND COMPUTER SCIENCE

Field of Study: Computer Science

Jakub Michalski

**Tile Defense – a tower defense game with a
procedurally generated world in OpenGL**

Engineer's Thesis is performed
in the Department of Mathematical Methods
in Computer Science (Informatics)
under supervision of
Ph.D. Eng. Bartosz Nowak

Olsztyn, 2026

Spis treści

Streszczenie	3
Abstract	4
Wstęp	5
0.1. Cel i zakres pracy	6
0.2. Założenia projektowe	6
0.3. Uzasadnienie zastosowania generowania proceduralnego	6
0.4. Rola szumu Perlina w warstwie wizualnej	7
0.5. Metodyka realizacji i weryfikacja działania	7
0.6. Struktura pracy	7
Rozdział 1. Część teoretyczna	8
Część teoretyczna	8
1.1. Wykorzystane technologie i biblioteki	8
1.2. Reprezentacja planszy i kodowanie typów pól	9
1.3. Generowanie proceduralne trasy	9
1.4. Szum Perlina jako podstawa generowania terenu	10
1.5. Wyznaczanie przeszkód na podstawie wartości szumu	10
1.6. Integracja z renderowaniem w OpenGL	11
1.6.1. Warstwa shaderów i zarządzanie programami	11
1.6.2. Renderowanie planszy jako siatki instancjonowanej	11
1.6.3. Przekazanie typów pól do GPU	11
1.6.4. Atlasy tekstur i dobór wariantu kafelka	11
1.6.5. Renderowanie obiektów dynamicznych i elementów interfejsu	12
Rozdział 2. Część praktyczna	13
2.1. Wymagania funkcjonalne i нефункционалне	13
2.1.1. Wymagania funkcjonalne	13
2.1.2. Wymagania нефункционалне	13
2.2. Architektura rozwiązania	14
2.2.1. Warstwy i główne moduły	14
2.2.2. Opis wybranych klas	14
2.2.3. Diagram klas	14
2.3. Diagramy funkcjonalne	17
2.3.1. Diagram przypadków użycia	17
2.3.2. Diagram aktywności dla ulepszania i sprzedaży wieży	17
2.3.3. Scenariusze użycia	18
2.3.4. Testy manualne	20
2.4. Implementacja kluczowych mechanizmów	21
2.4.1. Generowanie proceduralne mapy	21
2.4.2. Parametry wież i przeciwników	28
2.4.3. Przekazanie danych planszy do shaderów i renderowanie siatki	29

2.4.4.	Tekstura typów pól (<code>uTypes</code>)	31
2.4.5.	Wyróżnienie pola startowego i docelowego (<code>uStartCell</code> , <code>uGoalCell</code>)	35
2.4.6.	Losowe warianty kafelków (<code>uSeed</code>)	35
2.4.7.	Odświeżenie mapy podczas rozgrywki	35
2.4.8.	Struktura aplikacji i pętla główna gry	35
2.4.9.	Inicjalizacja zasobów renderowania	36
2.4.10.	Konfiguracja siatki i dopasowanie do obszaru okna	37
2.4.11.	Obsługa wejścia i wybór komórki	37
2.4.12.	Pętla główna i aktualizacja stanu gry	38
2.4.13.	Renderowanie sceny	38
2.4.14.	Zwalnianie zasobów	38
2.5.	Instrukcja użytkownika	38
2.5.1.	Sterowanie (podstawy)	38
2.5.2.	Menu główne	38
2.5.3.	Okno instrukcji <i>How to play?</i>	39
2.5.4.	Start rozgrywki i podział ekranu	40
2.5.5.	Panel sterowania bez zaznaczenia	40
2.5.6.	Wybór kafelka na mapie	41
2.5.7.	Budowa wieży	42
2.5.8.	Zaznaczenie wieży, ulepszenie i sprzedaż	44
2.5.9.	Uruchamianie fali i przeciwnicy	46
2.5.10.	Menu pauzy	46
2.5.11.	Zakończenie gry	47
Rozdział 3.	Podsumowanie	49
3.1.	Wnioski	49
3.2.	Możliwości rozbudowy	50
Spis rysunków		51
Spis tabel		52
Bibliografia		54

Streszczenie

Praca opisuje stworzenie gry typu *Tower Defense* w języku C++ z użyciem OpenGL 3.3 (*Core Profile*) w systemie Windows. Pokazuję cały proces budowy prostej gry 2D działającej w czasie rzeczywistym: jak jest ułożony program, jak działa rozgrywka, w jaki sposób wszystko jest rysowane na ekranie i jak wygląda interfejs użytkownika.

Najważniejszą częścią projektu jest proceduralne generowanie mapy. Trasa przeciwników powstaje automatycznie w taki sposób, żeby zawsze była poprawna i dało się nią przejść. Z kolei wygląd planszy oraz rozmieszczenie przeszkód są tworzone na podstawie dwuwymiarowego szumu Perlina. Dzięki temu można uzyskać wiele różnych układów mapy bez ręcznego projektowania każdego poziomu. W pracy opisuję, jak działa to generowanie i jak zostało wpięte w logikę gry.

Część praktyczna zawiera wymagania funkcjonalne i нефункционалне, opis architektury oraz najważniejszych mechanik, a także instrukcję obsługi ze zrzutami ekranu i proste testy działania. Na końcu opisano ograniczenia projektu, między innymi brak dźwięku, brak zapisu stanu gry, uproszczoną grafikę 2D oraz brak trwałego postępu między uruchomieniami.

Abstract

Tile Defense – a tower defense game with a procedurally generated world in OpenGL

This thesis describes the development of a *Tower Defense* game written in C++ using OpenGL 3.3 (Core Profile) on Windows. It walks through the full process of building a simple real-time 2D game: the program structure, gameplay logic, on-screen rendering, and the graphical user interface.

The main focus of the project is procedural map generation. The enemy path is created automatically in a way that keeps it valid and always traversable. The surrounding terrain and obstacle placement are generated with 2D Perlin noise. This makes it possible to produce many different map layouts without manually designing each level. The thesis explains how the generation works and how it is connected to the gameplay systems.

The practical part includes functional and non-functional requirements, an overview of the architecture and the key mechanics, a short user guide with screenshots, and basic tests. The thesis also lists the project limitations, including no audio, no game-state saving, simplified 2D visuals, and no persistent progression between runs.

Wstęp

Tworzenie gier komputerowych to dobry przykład projektu, w którym spotyka się kilka tematów naraz. Trzeba ogarnąć kod, który działa w czasie rzeczywistym, narysować coś na ekranie, obsłużyć wejście od użytkownika i dopilnować, żeby całość zachowywała się przewidywalnie. W praktyce szybko wychodzą też typowe problemy inżynierskie: jak podzielić program na części, jak nie pogubić się w zależnościach, jak utrzymać porządek w kodzie i jak sprawdzać, czy mechaniki działają tak jak trzeba.

Jednym z popularnych gatunków strategii jest *tower defense*. Zasada jest prosta: przeciwnicy idą po trasie do celu, a gracz stawia wieże, żeby ich zatrzymać. Na tej podstawie można jednak zbudować sporo logiki. Pojawia się zarządzanie zasobami, wybór typów wież, decyzja gdzie budować i kiedy ulepszać, a także planowanie fal. Wiele gier tego typu opiera świat na siatce pól, bo wtedy łatwiej kontrolować ruch, przeszkody, kolizje i zasady budowy.

Żeby gra nie była za każdym razem taka sama, potrzebna jest zmienność między kolejnymi uruchomieniami. Jeśli mapy są robione ręcznie, trzeba je projektować i testować jedna po drugiej, co zajmuje dużo czasu. Z tego powodu często stosuje się generowanie proceduralne, czyli automatyczne tworzenie zawartości na podstawie reguł i losowości. To może dotyczyć układu przeszkód, wyglądu planszy, przebiegu ścieżki albo detali wizualnych. Dobrze zaprojektowane generowanie daje wiele wariantów, a jednocześnie nie rozwała zasad gry i nie utrudnia orientacji na mapie.

W grach na siatce szczególnie ważna jest trasa przeciwników. Samo losowanie elementów nie gwarantuje, że da się przejść od startu do mety. Nawet jeśli przejście istnieje, przebieg trasy wpływa na balans, bo decyduje o tym, jak długo przeciwnik jest pod ostrzałem i ile jest miejsc na budowę wież. Dlatego losowość trzeba połączyć z kontrolą wyniku: weryfikacją poprawności i zestawem ograniczeń, które wycinają mapy niegrywalne albo skrajnie niewygodne.

Osobną sprawą jest wygląd mapy. W grafice często używa się metod, które dają „uporządkowaną losowość”, bo wtedy wynik wygląda bardziej naturalnie niż czysto losowe rozrzucenie. Jedną z takich metod jest szum Perlina, stosowany między innymi do terenów, tekstur czy efektów proceduralnych. Jego zaletą są płynne przejścia wartości, dzięki czemu tworzy struktury, które nie wyglądają jak przypadkowe plamy. W grze 2D można go wykorzystać do zróżnicowania tła i przeszkód, ale nadal trzeba pilnować czytelności, żeby gracz od razu widział trasę i pola ważne dla rozgrywki.

W projekcie istotny jest też wybór technologii. C++ daje dużą kontrolę nad tym, jak program działa i jak zarządza pamięcią, co jest ważne w aplikacjach czasu rzeczywistego. OpenGL to interfejs do renderingu 2D i 3D, który pozwala pracować na programowalnym potoku graficznym. W praktyce oznacza to shadery, tekstury, bufora wierzchołków i rysowanie sceny klatka po klatce. Taki zestaw dobrze pasuje do projektu, który ma pokazać zarówno część „logika gry”, jak i część „renderowanie”.

Z perspektywy edukacyjnej duża wartość polega na przejściu przez cały proces: od uruchomienia okna i kontekstu graficznego, przez pętlę gry, aż po spięcie aktualizacji stanu z

renderowaniem i interfejsem. To pomaga zrozumieć, co dzieje się w każdej klatce i dlaczego porządek w architekturze ma znaczenie, gdy projekt zaczyna rosnąć.

0.1. Cel i zakres pracy

Celem pracy jest zaprojektowanie i zaimplementowanie gry typu *tower defense* w języku C++ z wykorzystaniem OpenGL, z naciskiem na generowanie proceduralne. W projekcie zrealizowano dwa kluczowe mechanizmy:

- generowanie trasy przeciwników w taki sposób, aby mapa była poprawna i grywalna,
- generowanie krajobrazu na podstawie szumu Perlina, aby uzyskać zróżnicowanie wizualne.

Praca obejmuje część projektową, czyli zaplanowanie struktury programu i mechanik gry, oraz część implementacyjną, czyli stworzenie działającej aplikacji. Zakres obejmuje:

- przygotowanie zasad rozgrywki: fale przeciwników, budowę i ulepszanie wież, system zasobów oraz warunki zwycięstwa i porażki,
- implementację ruchu przeciwników po wygenerowanej trasie oraz interakcje z wieżami i pociskami,
- przygotowanie renderingu 2D w OpenGL: rysowanie mapy, jednostek i elementów interfejsu,
- integrację interfejsu użytkownika do sterowania rozgrywką i prezentacji stanu gry,
- przygotowanie dokumentacji: opis architektury i algorytmów, instrukcja użytkownika oraz testy.

0.2. Założenia projektowe

Projekt został przygotowany z myślą o uruchamianiu na komputerach osobistych z systemem Windows. Gra ma oprawę 2D, ponieważ w tym projekcie ważniejsza jest czytelność mapy, trasy i obiektów niż rozbudowana grafika. Świadomie ograniczono funkcje, które nie są kluczowe dla celu pracy, aby skupić się na generowaniu proceduralnym i implementacji mechanik.

Przyjęto następujące ograniczenia:

- brak dźwięku,
- brak zapisu i odczytu stanu gry,
- brak systemu postępu poziomów po zakończeniu rozgrywki,
- brak zapisu wygenerowanej mapy oraz odtwarzania jej na podstawie zadanego ziarna losowości.

Ograniczenia te nie blokują realizacji celu pracy, ale pokazują, co można rozbudować w kolejnych wersjach projektu.

0.3. Uzasadnienie zastosowania generowania proceduralnego

Generowanie proceduralne w grach ma kilka praktycznych zastosowań. Pozwala zwiększyć regrywalność, bo przy każdym uruchomieniu można dostać inny układ mapy. Pomaga też testować logikę gry, ponieważ zmieniające się środowisko szybciej ujawnia błędy w ruchu przeciwników, budowie wież i przebiegu fal. Dodatkowo samo generowanie może być osobnym problemem projektowym, gdzie najważniejsze jest połączenie losowości z regułami, które pilnują poprawnego wyniku.

W *tower defense* wynik generowania nie może być przypadkową mieszanką pól. Musi istnieć trasa od startu do mety, a układ przeszkód i tła nie może utrudniać orientacji. Ważne jest też ograniczenie skrajnych przypadków, na przykład trasy bardzo krótkiej albo bardzo długiej, bo to zmienia poziom trudności i balans. W praktyce oznacza to dwa kroki: zbudowanie mapy i sprawdzenie, czy spełnia warunki.

0.4. Rola szumu Perlina w warstwie wizualnej

Szum Perlina pozwala generować wartości, które zmieniają się płynnie w przestrzeni. W odróżnieniu od czystej losowości, gdzie sąsiednie pola mogą mocno się różnić, tutaj wynik daje struktury, które da się interpretować jako „obszary”. Dzięki temu szum może działać jak mapa gęstości albo tło do tworzenia tekstur.

W projekcie szum Perlina służy do różnicowania krajobrazu na mapie. Ma to dwa cele: dodać zmienność wizualną między rozgrywkami i jednocześnie nie zaszkodzić czytelności. W grze 2D szczególnie ważne jest wyraźne rozróżnienie trasy, tła i przeszkód, żeby gracz szybko rozumiał sytuację na planszy.

0.5. Metodyka realizacji i weryfikacja działania

Projekt był rozwijany iteracyjnie. Najpierw ustalono podstawowe wymagania i minimalny zestaw funkcji potrzebny do uruchomienia gry. Kolejnym krokiem było ułożenie struktury programu, tak aby oddzielić logikę rozgrywki (fale, przeciwnicy, wieże, pociski) od renderowania i interfejsu.

- Działanie rozwiązania sprawdzono testami funkcjonalnymi. Obejmują one między innymi:
- poprawność generowania mapy, w tym istnienie przejścia między punktem startowym i końcowym,
 - poprawność ruchu przeciwników po trasie oraz reakcję gry na dotarcie do celu,
 - poprawność budowy, ulepszania i sprzedaży wież,
 - poprawność działania pocisków, wykrywania trafień oraz naliczania nagród,
 - poprawność przebiegu fal oraz warunków zwycięstwa i porażki.

Efekty działania dokumentowano zrzutami ekranu i krótkimi opisami. Ułatwia to porównanie implementacji z założeniami oraz prezentację projektu w części praktycznej pracy.

0.6. Struktura pracy

Praca składa się z trzech rozdziałów, poprzedzonych niniejszym wstępem. Rozdział pierwszy to część teoretyczna: opis użytych technologii oraz omówienie podstaw działania generowania proceduralnego i szumu Perlina. Rozdział drugi to część praktyczna: wymagania, architektura, implementacja mechanik, instrukcja użytkowania oraz testy. Rozdział trzeci zawiera podsumowanie, wnioski oraz propozycje dalszego rozwoju projektu.

¹ Oryg.: „(semi-)automated creation of content through search algorithms.” Tłum. własne: „(pół-)automatyczne tworzenie zawartości za pomocą algorytmów przeszukiwania.” [1]

Rozdział 1

Część teoretyczna

1.1. Wykorzystane technologie i biblioteki

Projekt jest aplikacją desktopową napisaną w C++ i uruchamianą w systemie Microsoft Windows. Za rysowanie odpowiada OpenGL w wersji 3.3 (profil *Core*). W praktyce oznacza to korzystanie z programowalnego potoku renderowania, czyli shaderów wierzchołków i fragmen-tów w GLSL¹.

Do stworzenia okna, inicjalizacji kontekstu OpenGL oraz obsługi wejścia (klawiatura i mysz) użyto biblioteki GLFW². Ponieważ dostępność funkcji OpenGL zależy od sterowników i syste-mu, w projekcie zastosowano GLAD³, który ładuje wskaźniki do funkcji OpenGL potrzebnych aplikacji.

Interfejs użytkownika wykonano w Dear ImGui⁴. To podejście działa w trybie *immediate mode GUI*, więc dobrze pasuje do aplikacji renderowanych klatka po klatce. ImGui zostało zintegrowane z GLFW i OpenGL 3. Zgodnie z dokumentacją biblioteka przygotowuje bufory wierzchołków, które da się narysować w tym samym potoku renderowania co resztę sceny⁵.

Do wczytywania tekstur wykorzystano `stb_image`⁶. Obrazy są najpierw ładowane do pa-mięci operacyjnej, a później przesyłane do GPU jako tekstury używane przez shadery podczas rysowania planszy i obiektów gry.

Logika gry oraz struktury danych (w tym generowanie mapy, system fal i zachowanie wież, przeciwników oraz pocisków) powstały w oparciu o standardową bibliotekę C++. Dzięki temu projekt ma mało zależności zewnętrznych poza tym, co jest potrzebne do renderowania i UI.

Podczas projektowania struktury gry i pętli aktualizacji korzystano też z materiałów LearnOpenGL, w tym z rozdziału opisującego przykładową grę Breakout[7].

¹ Oryg.: „OpenGL is a software interface to graphics hardware.” Tłum. własne: „OpenGL jest programowym interfejsem do sprzętu graficznego.” [2, s. 15]

² Oryg.: „GLFW is an Open Source, multi-platform library for OpenGL, OpenGL ES and Vulkan develop-ment on the desktop.” Tłum. własne: „GLFW to otwartoźródłowa, wieloplatformowa biblioteka do tworzenia aplikacji desktopowych w oparciu o OpenGL, OpenGL ES i Vulkan.” [3]

³ Oryg.: „Multi-Language GL/GLES/EGL/GLX/WGL Loader-Generator based on the official specs.” Tłum. własne: „Wielojęzyczny generator loadera GL/GLES/EGL/GLX/WGL, oparty o oficjalne specyfikacje.” [4]

⁴ Oryg.: „Dear ImGui is a bloat-free graphical user interface library for C++.” Tłum. własne: „Dear ImGui to lekka biblioteka graficznego interfejsu użytkownika dla C++.” [5]

⁵ Oryg.: „It outputs optimized vertex buffers that you can render anytime in your 3D-pipeline-enabled appli-cation.” Tłum. własne: „Generuje zoptymalizowane bufory wierzchołków, które można renderować w dowolnym momencie w aplikacji posiadającej potok 3D.” [5]

⁶ Oryg.: „stb_image - v2.30 - public domain image loader”. Tłum. własne: „stb_image – v2.30 – moduł do wczytywania obrazów w domenie publicznej”. [6]

Autorstwo tekstur. Wszystkie tekstury wykorzystane w grze zostały przygotowane samodzielnie przez autora pracy. W projekcie nie wykorzystano zewnętrznych pakietów graficznych (assetów), dzięki czemu warstwa wizualna nie jest obciążona dodatkowymi ograniczeniami licencyjnymi.

1.2. Reprezentacja planszy i kodowanie typów pól

Plansza gry ma postać dwuwymiarowej siatki o rozmiarze $cols \times rows$. Typ każdego pola jest zapisany w jednowymiarowej tablicy indeksowanej wierszami. W implementacji zastosowano wartości całkowite z zakresu 0–255 (typ `unsigned char`). Wyróżniono trzy typy pól:

- wartość 0 oznacza tło (np. trawę),
- wartość 1 oznacza trasę (drogę), po której poruszają się przeciwnicy,
- wartość 2 oznacza przeszkodę, czyli pole blokujące budowę lub ruch.

Żeby shader mógł szybko sprawdzić typ pola podczas rysowania, kody pól są przesyłane do GPU jako tekstura dwuwymiarowa z jednym kanałem (format `GL_R8`). Dzięki temu shader fragmentów odczytuje typ komórki i na tej podstawie wybiera właściwy fragment atlasu tekstur.

1.3. Generowanie proceduralne trasy

Trasa przeciwników jest generowana automatycznie tak, aby każda nowa mapa była poprawna i nadawała się do gry bez ręcznego układania poziomu. Trasa powstaje jako ciąg sąsiadujących komórek siatki, przy ruchu w czterech kierunkach: góra, dół, lewo, prawo. Wynik jest przechowywany jako uporządkowana lista współrzędnych, z której później korzystają przeciwnicy podczas poruszania się.

Punkt startowy jest losowany na lewej krawędzi planszy (kolumna $x = 0$). Jeżeli wymiary na to pozwalają, pomijane są skrajne wiersze, żeby start nie wypadł w samym rogu. Punkt końcowy (meta) jest wybierany losowo w jednym z dwóch wariantów: na prawej krawędzi planszy ($x = cols - 1$) z losowym położeniem w osi y albo na krawędzi przeciwnej w osi y z losowym położeniem w osi x w prawej części planszy. W praktyce, w zależności od przyjętej konwencji osi (np. odwrócenia osi y w teksturach), wariant ten może odpowiadać krawędzi widocznej w dolnej części ekranu.

Sama generacja działa jako iteracyjny spacer po siatce. Podejście jest powtarzane wiele razy w ograniczonej liczbie prób. W każdej próbie utrzymywana jest tablica odwiedzin, aby nie wracać na wcześniej użyte pole, oraz lista pól należących do aktualnie budowanej trasy. Dodano też ograniczenia, które pilnują geometrii trasy i zapobiegają układom problematycznym w grze. W szczególności:

- nie można wejść ponownie na odwiedzoną komórkę,
- nie można tworzyć pełnych bloków 2×2 złożonych wyłącznie z pól trasy, aby nie powstawały „kwadraty”,
- nowe pole ma stykać się z dotychczasową trasą tylko w jednym miejscu (z poprzednim krokiem), co ogranicza rozgałęzienia i boczne „przyklejenia”.

Wybór kolejnego kroku wspiera prosta heurystyka oparta o odległość Manhattan do punktu końcowego. Dla każdego kandydata liczona jest wartość:

$$d = |x - x_{cel}| + |y - y_{cel}|,$$

a następnie ruchy są dzielone na te, które zmniejszają odległość do celu, oraz te, które ją zwiększają. Z ustalonym prawdopodobieństwem preferowane są kroki przybliżające do mety,

ale czasem dopuszczany jest też krok „w gorszą stronę”. To pomaga wyjść z sytuacji, w których trasa utknęłaby bez wyjścia, i daje bardziej różne przebiegi niż zawsze najkrótsza ścieżka.

7

Każda próba ma limit liczby kroków, a cały proces ma limit prób (w implementacji przyjęto 200 prób). Wprowadzono też minimalną długość trasy, aby odrzucać mapy, na których przeciwnicy doszliby do mety zbyt szybko. Jeżeli w zadanej liczbie prób nie uda się znaleźć trasy spełniającej warunki, używany jest wariant awaryjny: prosta droga przechodząca przez środkowy wiersz planszy. Dzięki temu gra zawsze dostaje poprawną planszę wejściową.

1.4. Szum Perlina jako podstawa generowania terenu

Do zróżnicowania tła planszy użyto szumu Perlina. To funkcja pseudolosowa, która nie losuje każdej komórki niezależnie, tylko tworzy wartości zmieniające się płynnie w przestrzeni. W efekcie sąsiednie pola mają zbliżone wartości, więc zamiast przypadkowej „mozaiki” pojawiają się większe obszary o podobnym charakterze, na przykład skupiska przeszkód.

8

W projekcie używany jest wariant dwuwymiarowy. Dla punktu (x, y) obliczana jest wartość $n(x, y)$ z przedziału $[0, 1]$. Wartość powstaje na podstawie funkcji wygładzającej oraz interpolacji między wartościami w węzłach siatki szumu. To sprawia, że wynik jest ciągły i nadaje się do tworzenia struktur, które wyglądają „naturalniej” niż czysty los.

Dla większej różnorodności zastosowano sumowanie kilku warstw szumu, czyli fraktalny szum Perlina (ang. *fractal Brownian motion*). W praktyce liczone jest kilka „oktaw” o rosnącej częstotliwości i malejącej amplitudzie, a potem wynik jest uśredniany. Dzięki temu parametrami można sterować charakterem tła, na przykład tym, czy przeszkody mają tworzyć duże obszary, czy raczej drobniejszy wzór.

1.5. Wyznaczanie przeszkód na podstawie wartości szumu

Po wyznaczeniu głównej trasy między startem a metą uzupełniane są pozostałe pola planszy. Dla każdej komórki, która nie należy do trasy, obliczana jest wartość szumu $n(x, y) \in [0, 1]$. Następnie wykonywane jest progowanie, czyli porównanie z progiem T :

- jeżeli $n(x, y) > T$, komórka staje się przeszkodą,
- w przeciwnym wypadku komórka zostaje tłem.

To jest proste do wyjaśnienia: szum tworzy „mapę wartości”, a próg mówi, które miejsca mają zostać zablokowane. Ważne jest też to, że pola należące do trasy nie są modyfikowane, więc przechodniość mapy zostaje zachowana.

Żeby kolejne mapy różniły się od siebie nawet przy stałych parametrach, w implementacji użyto losowego przesunięcia współrzędnych wejściowych (tzw. *shift*) oraz skali szumu. Dzięki temu dla tych samych indeksów komórek (x, y) szum jest próbkowany w innym miejscu, co daje inny układ przeszkód. Skala wpływa na „rozmiar plam”: mniejsza skala daje większe, zwarte obszary, a większa skala tworzy drobniejsze zróżnicowanie.

⁷ Odległość Manhattan (metryka taksówkowa, L^1) jest zdefiniowana jako suma wartości bezwzględnych różnic współrzędnych. W przypadku ruchu po siatce w czterech kierunkach odpowiada długości najkrótszej ścieżki prowadzonej wzdłuż osi siatki. [8, 9]

⁸ Oryg.: „Procedural noise is an important building block for visually rich rendering.” Tłum. własne: „Szum proceduralny jest istotnym elementem składowym wizualnie bogatego renderowania.” [10]

Na etapie renderowania typom pól przypisywane są warianty tekstur. To pomaga uniknąć wrażenia, że tło i przeszkody są powielane jako jeden kafelek w kółko.

1.6. Integracja z renderowaniem w OpenGL

Renderowanie gry zrealizowano w oparciu o OpenGL 3.3 (profil *core*). Okno i kontekst tworzy GLFW, a dostęp do funkcji OpenGL zapewnia GLAD. Taki zestaw jest typowy w projektach tego typu i pozwala jasno określić, z jakich elementów API korzysta aplikacja.

1.6.1. Warstwa shaderów i zarządzanie programami

Do obsługi shaderów przygotowano klasę pomocniczą. Jej zadaniem jest wczytanie kodu shaderów, kompilacja shaderów wierzchołków i fragmentów oraz połączenie ich w program renderujący. Jeżeli pojawi się błąd kompilacji albo linkowania, zapisywany jest komunikat diagnostyczny i aplikacja przerywa działanie. W praktyce to oszczędza czas, bo błąd w shaderze od razu jest widoczny i opisany.

1.6.2. Renderowanie planszy jako siatki instancjonowanej

Plansza jest regularną siatką kafelków, więc do rysowania zastosowano instancjonowanie. Geometrią bazową jest pojedynczy czworokąt złożony z dwóch trójkątów, opisany w znormalizowanych współrzędnych urządzenia (ang. *Normalized Device Coordinates*). Zamiast wykonywać osobne wywołanie rysowania dla każdego kafelka, rysowana jest jedna geometria wiele razy, a liczba instancji odpowiada liczbie pól siatki.

W shaderze wierzchołków pozycja kafelka jest wyliczana na podstawie identyfikatora instancji. Ten identyfikator jest zamieniany na współrzędne komórki (*col, row*), a potem wyliczane jest przesunięcie wynikające z rozmiaru kafelka, odstępów oraz położenia całej siatki na ekranie. Dzięki temu plansza jest ustawiana spójnie, a jej rozmiar da się łatwo dopasować do okna.

1.6.3. Przekazanie typów pól do GPU

Po stronie procesora przechowywane są typy pól. Żeby shader mógł z nich korzystać, kody pól są synchronizowane do tekstury (format *GL_R8*) o rozmiarze równym siatce. Każdy piksel tej tekstury odpowiada jednej komórce i trzyma kod typu pola. Shader fragmentów może to odczytać funkcją `texelFetch`, bez przesyłania dużej ilości danych jako uniformy.

1.6.4. Atlasy tekstur i dobór wariantu kafelka

Wygląd planszy opiera się na atlasach tekstur, czyli obrazach z wieloma kafelkami w jednym pliku. Zastosowano osobne atlasy dla tła, drogi, pola startowego, pola docelowego oraz przeszkód. W shaderze fragmentów najpierw ustalany jest typ pola, potem wybierany atlas, a na końcu wyliczane są współrzędne fragmentu atlasu odpowiadającego konkretnemu kafelkowi.

Dla tła i przeszkód wariant kafelka jest wybierany prostą funkcją mieszającą zależną od współrzędnych komórki oraz ziarna losowości. Ziarno jest ustawiane przy uruchomieniu rozgrywki oraz przy generowaniu nowej mapy. Dzięki temu jedna mapa jest deterministyczna, ale kolejne mapy mogą wyglądać inaczej.

Dla drogi wariant zależy od sąsiedztwa. Shader sprawdza typy pól w czterech kierunkach (góra, dół, lewo, prawo), a potem dobiera odpowiedni kafelek, na przykład odcinek poziomy,

pionowy albo zakręt. Pozwala to narysować ciągłą ścieżkę bez przechowywania osobnych danych o jej kształcie. Pola startu i mety są wyróżniane przez porównanie aktualnej komórki z pozycjami przekazanymi jako uniformy.

Dla zachowania estetyki grafiki pikselowej ustawiono filtrowanie **nearest** oraz tryb zawijania **clamp-to-edge**. Dzięki temu tekstury nie rozmywają się przy skalowaniu i nie pojawiają się artefakty na krawędziach atlasu.

1.6.5. Renderowanie obiektów dynamicznych i elementów interfejsu

Po wyrenderowaniu planszy rysowane są obiekty dynamiczne: przeciwnicy, wieże i pociski. Każda grupa ma osobny program shaderów dopasowany do sposobu prezentacji. Przeciwnicy i wieże są rysowane jako czworokąty z teksturą, a przezroczystość jest obsługiwana przez odrzucenie fragmentów o niskiej wartości alfa. Pociski oraz wskaźniki pomocnicze, na przykład okrąg zasięgu, są generowane w shaderze fragmentów na podstawie odległości od środka obiektu, więc nie wymagają osobnych tekstur.

Interfejs użytkownika wykonano w Dear ImGui. Panel jest rysowany niezależnie od planszy i pozwala pokazać parametry rozgrywki, uruchamiać fale oraz zarządzać wieżami. W renderingu działa to jako warstwa nakładana na scenę po narysowaniu elementów gry.

Rozdział 2

Część praktyczna

2.1. Wymagania funkcjonalne i нефункционалне

Aplikację przygotowano jako desktopową grę typu *Tower Defense*. Projekt uruchamiany jest w systemie Microsoft Windows (Windows 10/11) i wymaga obsługi OpenGL w wersji 3.3 (profil *Core*). Interfejs oraz komunikaty wyświetlane są w języku angielskim.

2.1.1. Wymagania funkcjonalne

- Uruchomienie rozgrywki z menu głównego (Start game) oraz zakończenie działania aplikacji (Exit game).
- Wyświetlenie instrukcji obsługi w formie okna dialogowego (How to play?).
- Rozgrzywka podzielona na 10 kolejnych fal przeciwników. Gracz uruchamia falę ręcznie przyciskiem *Next wave*.
- Budowa trzech typów wież: *Basic*, *Slow* oraz *Fast*. Wieże mogą być stawiane wyłącznie na kafelkach niebędących drogą oraz bez przeszkód.
- Obsługa kliknięć myszą w obszar mapy (wybór kafelka / wieży) oraz przycisków panelu sterowania.
- Przeciwnicy poruszają się po wyznaczonej drodze od punktu startowego do celu. Dotarcie przeciwnika do celu powoduje utratę punktów życia (HP).
- Warunek zwycięstwa: ukończenie ostatniej fali.
- Warunek porażki: spadek liczby punktów życia (HP) do zera.
- Po zakończeniu gry (zwycięstwo lub porażka) wyświetlane jest okno z opcjami ponownego rozpoczęcia (Replay/Try again), wygenerowania nowej mapy (New map) oraz powrotu do menu głównego (Main menu).
- Menu w trakcie rozgrywki (pauza): *Resume*, *Restart*, *New map*, *How to play?*, *Go back to main menu*.

2.1.2. Wymagania нефункционалне

- Aplikacja przeznaczona do uruchamiania w środowisku Windows 10/11.
- Renderowanie w czasie rzeczywistym z wykorzystaniem OpenGL 3.3 (profil *Core*).
- Stabilna praca w typowych rozdzielczościach okna aplikacji; układ interfejsu dopasowuje się do rozmiaru okna.
- W ramach projektu nie zaimplementowano systemu dźwięku, zapisu stanu gry ani wyboru poziomu trudności.

2.2. Architektura rozwiązania

Kod rozdzielono na części odpowiadające głównym obszarom działania gry: stan i reguły rozgrywki, symulacja przeciwników i pocisków, mapa z generowaniem proceduralnym, renderowanie oraz interfejs użytkownika. Taki układ porządkuje projekt. Łatwiej wskazać, gdzie powstaje dany efekt w grze i gdzie należy szukać błędu, gdy coś przestaje działać.

2.2.1. Warstwy i główne moduły

- **Warstwa logiki gry** - przechowuje stan rozgrywki i reguły, między innymi punkty życia, złoto, numer fali, stan zakończenia gry, listę wież oraz informacje o aktualnym zaznaczeniu.
- **Warstwa symulacji przeciwników i pocisków** - odpowiada za uruchamianie fal, aktualizację przeciwników w czasie oraz obsługę ataku wież i działania pocisków.
- **Warstwa mapy** - przechowuje siatkę kafelków i typy pól oraz realizuje generowanie proceduralne, w tym trasę przeciwników i rozmieszczenie przeszkód.
- **Warstwa renderowania** - przygotowuje zasoby GPU, kompiluje shadery i odpowiada za rysowanie mapy oraz obiektów.
- **Warstwa interfejsu użytkownika** - realizuje menu, panel sterowania oraz obsługę zdarzeń myszy w kontekście wyboru kafelka i wieży.

2.2.2. Opis wybranych klas

- **GameState** - struktura przechowująca kluczowe dane rozgrywki, takie jak złoto, HP, numer fali, flagi zwycięstwa i porażki, lista wież oraz bieżący wybór.
- **Grid** - reprezentacja mapy jako siatki. Klasa przechowuje typy pól, generuje trasę i przeszkody oraz udostępnia metody do sprawdzania możliwości budowy na wybranej komórce.
- **WaveSystem** - moduł odpowiedzialny za definicję fal oraz sterowanie ich przebiegiem, w tym przejścia stanów.
- **ProjectileSystem** - moduł odpowiedzialny za symulację pocisków oraz wykrywanie trafień.
- **Shader** - klasa narzędziowa upraszczająca kompilację i linkowanie shaderów oraz ustawianie parametrów (*uniforms*).

2.2.3. Diagram klas

W tym miejscu krótko opisano role klas, z których składa się implementacja gry.

Klasa **GameTypes** porządkuje wspólne typy używane w całym projekcie (na przykład typ pola, typ wieży i typ przeciwnika). Trzyma też struktury ze statystykami oraz funkcje pomocnicze, które zwracają bazowe parametry jednostek i wykonują proste obliczenia używane w ekonomii gry.

Klasa **Grid** reprezentuje mapę jako siatkę pól oraz przechowuje informacje o trasie przeciwników. Odpowiada za generowanie proceduralne poziomu, a także udostępnia metody do sprawdzania, czy na danym polu można budować i do przeliczania współrzędnych ekranu na współrzędne siatki.

Struktura **GameState** przechowuje bieżący stan rozgrywki, czyli między innymi złoto, punkty życia, numer fali oraz flagi zwycięstwa i porażki. Zawiera też listę postawionych wież i informacje o aktualnym zaznaczeniu, dzięki czemu inne moduły mogą szybko sprawdzić, co dzieje się w grze.

Struktura Tower opisuje pojedynczą wieżę postawioną na mapie. Przechowuje jej typ, pozycję w siatce, poziom oraz kwotę zainwestowanego złota, na podstawie której wyliczana jest między innymi wartość sprzedaży.

Struktura WaveConfig definiuje parametry jednej fali przeciwników. Zawiera typ przeciwnika, liczbę jednostek i tempo ich pojawiania się, a także współczynniki wpływające na trudność oraz nagrodę za pokonanie.

Klasa WaveSystem steruje przebiegiem fal i utrzymuje listę aktualnie żywych przeciwników. W trakcie rozgrywki na podstawie konfiguracji fali „wypuszcza” kolejne jednostki w czasie, aktualizuje ich ruch po trasie i sygnalizuje, czy fala jest aktywna oraz czy została zakończona.

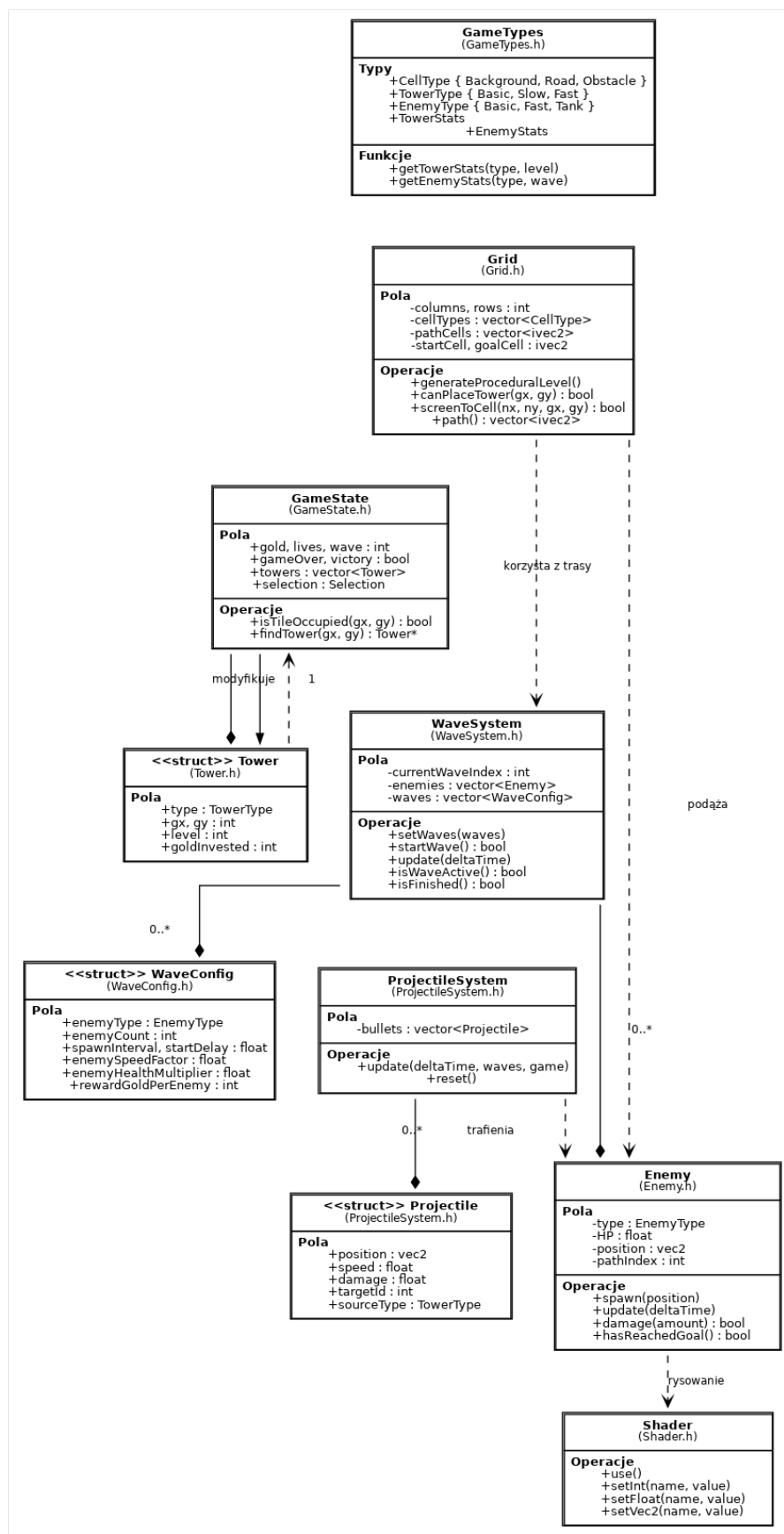
Klasa Enemy reprezentuje pojedynczego przeciwnika poruszającego się po ścieżce. Przechowuje jego typ, punkty życia, pozycję oraz indeks bieżącego kroku na trasie, a w aktualizacji przesuwa jednostkę do kolejnych punktów i pozwala stwierdzić, czy dotarła do celu.

Struktura Projectile opisuje pojedynczy pocisk wystrzelony przez wieżę. Zawiera pozycję, prędkość i obrażenia, a także identyfikator celu, dzięki czemu może „podążać” za konkretnym przeciwnikiem.

Klasa ProjectileSystem zarządza wszystkimi aktywnymi pociskami w grze. W każdej aktualizacji przesuwa pociski w kierunku celów, sprawdza trafienia i usuwa te, które zakończyły swoje działanie, a także generuje nowe strzały na podstawie wież znajdujących się w **GameState** i przeciwników z **WaveSystem**.

Klasa Shader upraszcza pracę z shaderami w OpenGL. Odpowiada za wczytanie kodu, kompilację i linkowanie programu shaderów oraz za jego aktywację podczas renderowania.

Zależności pomiędzy opisanymi elementami zostały podsumowane na diagramie klas (Rys. 2.1).



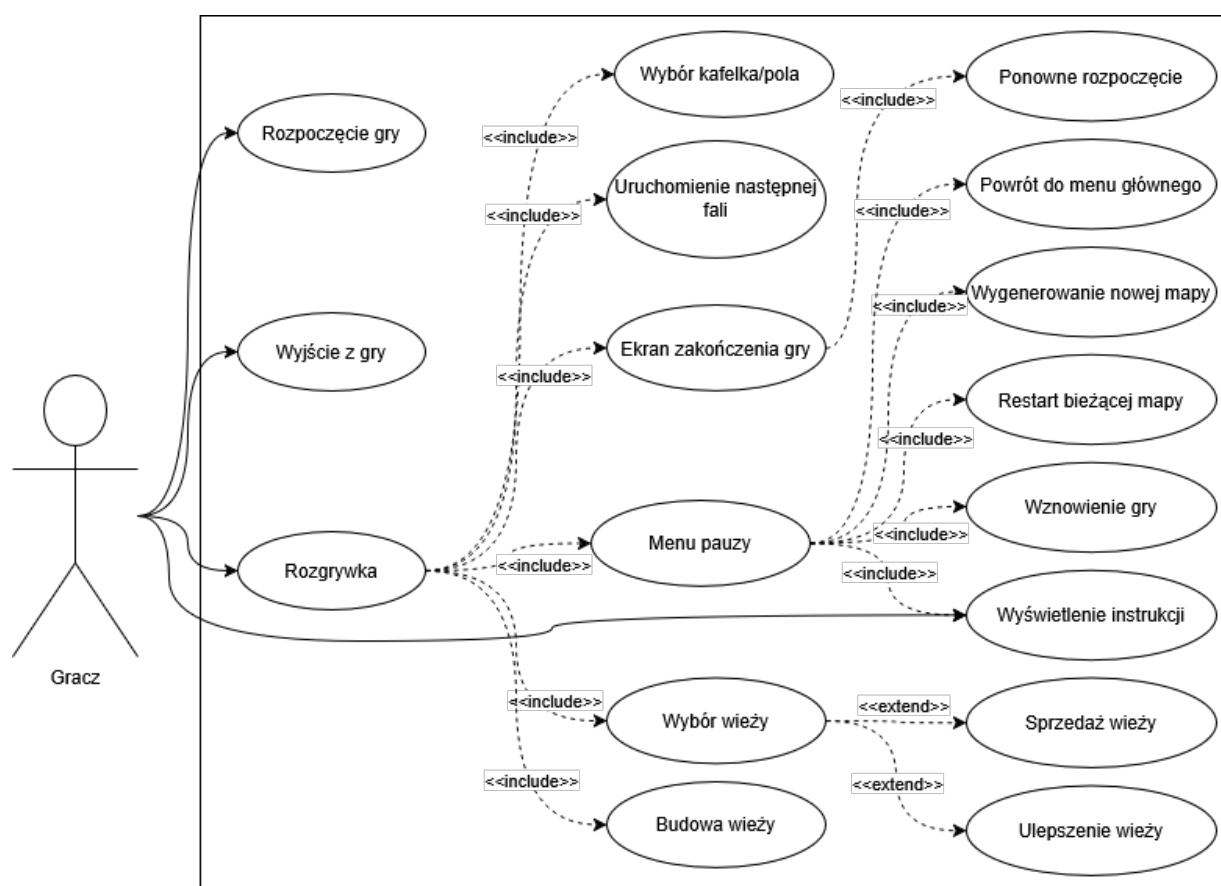
Rysunek 2.1. Diagram klas przedstawiający kluczowe elementy architektury gry.

2.3. Diagramy funkcjonalne

Aby doprecyzować zachowanie aplikacji od strony gracza, przygotowano diagramy pokazujące dostępne funkcje oraz przebieg wybranego procesu w grze.

2.3.1. Diagram przypadków użycia

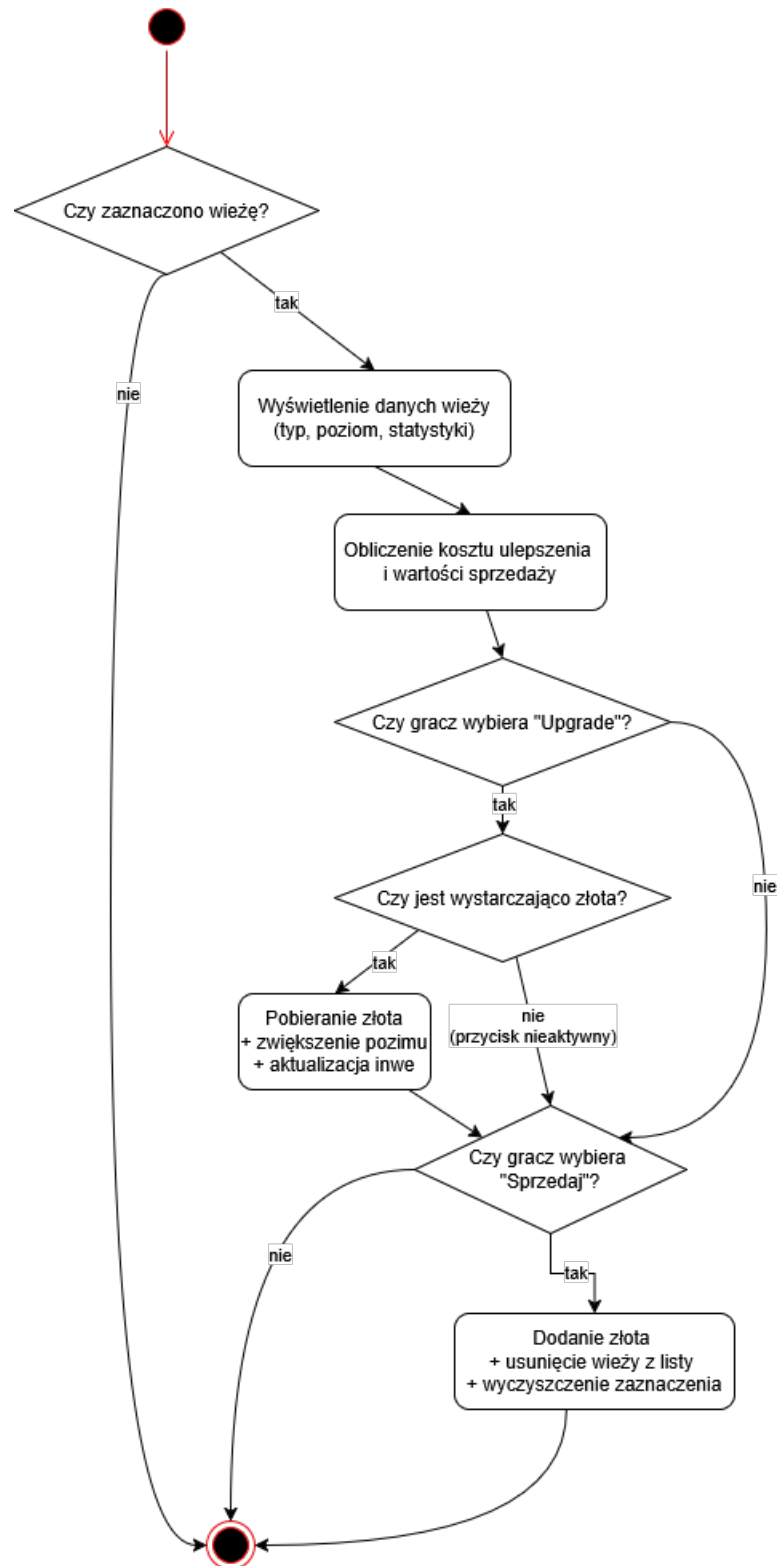
Zakres funkcji dostępnych dla gracza oraz relacje pomiędzy nimi ujęto w diagramie przypadków użycia (Rys. 2.2). Obejmuje on start rozgrywki, pauzę, uruchamianie fal, budowę i zarządzanie wieżami oraz zakończenie gry. Relacja «include» wskazuje kroki wykonywane w ramach większych przypadków użycia, natomiast «extend» opisuje działania opcjonalne, takie jak sprzedaż i ulepszanie wieży.



Rysunek 2.2. Diagram przypadków użycia gry.

2.3.2. Diagram aktywności dla ulepszania i sprzedaży wieży

Po zaznaczeniu wieży aplikacja realizuje sekwencję kroków związaną z prezentacją parametrów, wyliczeniem kosztu ulepszenia i wartości sprzedaży oraz weryfikacją stanu złota (Rys. 2.3). Najpierw prezentowane są parametry wieży, następnie obliczany jest koszt ulepszenia i wartość sprzedaży. Kolejny krok to sprawdzenie stanu złota. Jeżeli środków wystarcza, wykonywane jest ulepszenie. W przypadku sprzedaży wieża jest usuwana z listy obiektów, a zaznaczenie zostaje wyczyszczone.



Rysunek 2.3. Diagram aktywności: ulepszanie i sprzedaż wieży.

2.3.3. Scenariusze użycia

Poniższe scenariusze pokazują typowe sytuacje w grze. Opisują kolejność działań gracza oraz to, jak reaguje aplikacja.

Uruchomienie następnej fali

Scenariusz uruchomienia następnej fali opisuje kolejne akcje gracza oraz reakcje systemu (Tabela 2.1).

Tabela 2.1. Scenariusz użycia: uruchomienie następnej fali.

Krok	Akcja użytkownika	Reakcja systemu
1	Gracz znajduje się w ekranie rozgrywki.	System wyświetla panel sterowania z przyciskiem <i>Next wave</i> oraz informacjami o stanie gry.
2	Gracz wybiera <i>Next wave</i> .	System inicjalizuje parametry fali oraz uruchamia generowanie przeciwników w czasie zgodnie z konfiguracją.
3	–	Przeciwnicy pojawiają się na polu startowym w odstępach czasowych; stan fali przechodzi w tryb aktywny.
4	Gracz podejmuje działania w trakcie fali (np. budowa/ulepszenie wież).	System aktualizuje logikę gry (ruch przeciwników, obsługa pocisków, ekonomia) i odświeża prezentowane informacje.
5	–	Po wygenerowaniu wszystkich jednostek i pokonaniu ostatniego przeciwnika system oznacza falę jako zakończoną.
6	Gracz ponownie wybiera <i>Next wave</i> .	System uruchamia kolejną falę, o ile nie został spełniony warunek zwycięstwa lub porażki.

Ulepszenie i sprzedaż wieży

Scenariusz ulepszenia i sprzedaży opisuje operacje dostępne po zaznaczeniu wieży oraz warunki ich wykonania (Tabela 2.2).

Tabela 2.2. Scenariusz użycia: ulepszenie lub sprzedaż zaznaczonej wieży.

Krok	Akcja użytkownika	Reakcja systemu
1	Gracz wskazuje na mapie kafelek z wieżą.	System zaznacza wieżę i wyświetla jej parametry (typ, poziom, statystyki) w panelu sterowania.
2	–	System oblicza koszt ulepszenia oraz wartość sprzedaży na podstawie parametrów wieży i wyświetla je użytkownikowi.
3	Gracz wybiera <i>Upgrade tower</i> .	Jeżeli posiadana ilość złota jest wystarczająca, system odejmuje koszt, zwiększa poziom wieży i aktualizuje zainwestowane złoto.
4	Gracz próbuje ulepszyć wieżę przy niewystarczającej ilości złota.	System blokuje operację, a przycisk ulepszenia pozostaje nieaktywny.
5	Gracz wybiera <i>Sell tower</i> .	System dodaje wartość sprzedaży do stanu złota, usuwa wieżę z listy obiektów i czyści zaznaczenie.
6	–	Odświeżone zostają informacje w panelu sterowania; dalsze działania dotyczą nowego zaznaczenia lub wyboru kafelka.

2.3.4. Testy manualne

Sprawdzenie działania aplikacji wykonano w formie testów manualnych. Skupiono się na funkcjach, które bezpośrednio wpływają na przebieg rozgrywki, oraz na sytuacjach powiązanych z ekonomią i stanami gry. Wyniki potwierdziły zgodność implementacji z wymaganiami funkcjonalnymi. Przykładowe testy manualne kluczowych funkcji, wraz z oczekiwanymi rezultatami, zestawiono w tabeli 2.3.

Tabela 2.3. Przykładowe testy manualne kluczowych funkcji aplikacji.

ID	Opis testu	Oczekiwany rezultat
T1	Uruchomienie rozgrywki z menu głównego (<i>Start game</i>).	Wygenerowanie mapy i inicjalizacja stanu gry.
T2	Próba postawienia wieży na polu drogi.	Operacja zablokowana; brak dodania wieży.
T3	Uruchomienie fali przyciskiem <i>Next wave</i> .	Start fali i generowanie przeciwników w czasie.
T4	Dotarcie przeciwnika do pola docelowego.	Zmniejszenie HP oraz usunięcie przeciwnika.
T5	Ulepszenie wieży przy wystarczającej ilości złota.	Zmniejszenie złota; wzrost poziomu i statystyk.
T6	Ulepszenie wieży przy niewystarczającej ilości złota.	Przycisk ulepszenia nieaktywny; brak zmian.
T7	Sprzedaż wieży.	Zwrot części złota; usunięcie wieży; wyczyszczenie zaznaczenia.
T8	Spełnienie warunku zwycięstwa lub porażki.	Wyświetlenie odpowiedniego ekranu zakończenia gry.

2.4. Implementacja kluczowych mechanizmów

W tej sekcji opisano mechanizmy, które mają bezpośredni wpływ na działanie gry. Uwzględniono elementy renderowania i przekazywania danych do shaderów oraz fragmenty logiki rozgrywki, między innymi generowanie mapy, system fal, zachowanie przeciwników i działanie wież. Fragmenty kodu źródłowego przedstawiono z użyciem środowiska `listings`. W celu zachowania zgodności z implementacją listingi są wczytywane bezpośrednio z plików projektu wraz z podaniem zakresów linii.

2.4.1. Generowanie proceduralne mapy

Mapę generowano proceduralnie, aby każda rozgrywka mogła wyglądać inaczej bez ręcznego projektowania poziomów. Plansza ma postać siatki kafelków, a każdy kafelek reprezentuje tło, drogę albo przeszkodę. Generowanie przebiega w dwóch krokach:

- najpierw wyznaczana jest przechodnia trasa przeciwników od punktu startowego do punktu docelowego,
- następnie na polach poza trasą rozmieszczane są przeszkody w układzie przypominającym skupiska terenu.

Takie podejście zapewnia, że ścieżka zawsze istnieje, a jednocześnie układ planszy zmienia się między uruchomieniami.

Proceduralne wyznaczanie trasy przeciwników

Trasa powstaje jako sekwencja kroków po sąsiednich polach siatki. Start znajduje się w punkcie wejścia, a kolejne pole wybierane jest w jednym z czterech kierunków (góra, dół, lewo, prawo), aż do osiągnięcia mety. Aby trasa była poprawna i nie sprawiała problemów w dalszych etapach gry, przyjęto ograniczenia konstrukcyjne:

- zakaz ponownego wejścia na już odwiedzone pole, co eliminuje pętle,
- zakaz tworzenia pełnych kwadratów 2×2 z pól drogi,
- limity liczby kroków i liczby prób, a w przypadku braku poprawnego wyniku zastosowanie wariantu awaryjnego.

Ograniczenie dotyczące układu 2×2 wynika ze sposobu dobierania tekstur drogi na podstawie sąsiedztwa. Przy pełnych blokach drogi wybór kafelków potrafił „mieszać” połączenia. W praktyce pojawiały się zakręty sugerujące boczne odgałęzienia, mimo że logicznie ich nie było. Eliminacja układu 2×2 upraszcza geometrię trasy i stabilizuje wygląd drogi.

Dodatkowo preferowane są ruchy zmniejszające odległość do mety, przy zachowaniu losowości, aby przebiegi tras nie były identyczne. Implementacja wyznaczania trasy, wraz z obsługą prób generacji i reguł poprawności, znajduje się w metodzie `Grid::generateProceduralLevel` (Listing 2.1).

Listing 2.1. Etap 1: wyznaczanie trasy (start/meta, próby generacji, reguły poprawności trasy) w `Grid::generateProceduralLevel` (fragment `Grid.cpp`).

```

1 void Grid::generateProceduralLevel(bool uploadTexture)
2 {
3     int cols = params_.cols;
4     int rows = params_.rows;
5     if (cols <= 0 || rows <= 0)
6         return;
7
8     auto index = [cols](int x, int y)
9     {
10         return y * cols + x;
11     };
12
13     auto wouldCreateSquare = [&](int nx, int ny,
14         const std::vector<unsigned char>& visited) -> bool
15     {
16         if (cols < 2 || rows < 2) return false;
17
18         for (int tx = nx - 1; tx <= nx; ++tx)
19         {
20             for (int ty = ny - 1; ty <= ny; ++ty)
21             {
22                 if (tx < 0 || ty < 0 || tx + 1 >= cols || ty + 1 >= rows)
23                     continue;
24
25                 int idx00 = index(tx, ty);
26                 int idx10 = index(tx + 1, ty);
27                 int idx01 = index(tx, ty + 1);
28                 int idx11 = index(tx + 1, ty + 1);
29
30                 auto isVisitedOrNew = [&](int cx, int cy, int idx)
31                 {
32                     if (cx == nx && cy == ny)
33                         return true;
34                     return visited[idx] != 0;
35                 };

```



```

36         bool v00 = isVisitedOrNew(tx, ty, idx00);
37         bool v10 = isVisitedOrNew(tx + 1, ty, idx10);
38         bool v01 = isVisitedOrNew(tx, ty + 1, idx01);
39         bool v11 = isVisitedOrNew(tx + 1, ty + 1, idx11);
40
41         if (v00 && v10 && v01 && v11)
42             return true;
43     }
44 }
45 return false;
46 };
47
48
49 std::fill(types_.begin(), types_.end(), 0);
50 mainPath_.clear();
51
52 static std::mt19937 rng(std::random_device{}());
53
54
55 std::uniform_real_distribution<float> noiseShiftDist(0.0f, 1000.0f);
56 float noiseShiftX = noiseShiftDist(rng);
57 float noiseShiftY = noiseShiftDist(rng);
58
59 const int maxAttempts = 200;
60 const int maxSteps = cols * rows;
61 const int minPathLen = (cols + rows) / 2;
62
63 for (int attempt = 0; attempt < maxAttempts; ++attempt)
64 {
65     std::vector<unsigned char> visited(cols * rows, 0);
66     std::vector<std::pair<int, int>> tmpPath;
67     tmpPath.reserve(cols * rows);
68
69     int startYMin = (rows > 2) ? 1 : 0;
70     int startYMax = (rows > 2) ? rows - 2 : rows - 1;
71     std::uniform_int_distribution<int> startRowDist(startYMin, startYMax);
72
73     int sx = 0;
74     int sy = startRowDist(rng);
75
76     int exitSide = 0;
77     if (rows > 2)
78     {
79         std::uniform_int_distribution<int> sideDist(0, 1);
80         exitSide = sideDist(rng);
81     }
82
83     int ex = cols - 1;
84     int ey = sy;
85
86     if (exitSide == 0)

```

```

87     {
88         std::uniform_int_distribution<int> exitRowDist(startYMin, startYMax);
89         ex = cols - 1;
90         ey = exitRowDist(rng);
91     }
92     else
93     {
94         int minExitX = cols / 2;
95         std::uniform_int_distribution<int> exitColDist(minExitX, cols - 1);
96         ex = exitColDist(rng);
97         ey = 0;
98     }
99
100     int x = sx;
101     int y = sy;
102     visited[index(x, y)] = 1;
103     tmpPath.push_back({ x, y });
104
105     bool reachedExit = false;
106     std::uniform_real_distribution<float> prob01(0.0f, 1.0f);
107
108     for (int step = 0; step < maxSteps; ++step)
109     {
110         if (exitSide == 0)
111         {
112             if (x == cols - 1 && (int)tmpPath.size() >= minPathLen)
113             {
114                 reachedExit = true;
115                 break;
116             }
117         }
118         else
119         {
120             if (x == ex && y == 0 && (int)tmpPath.size() >= minPathLen)
121             {
122                 reachedExit = true;
123                 break;
124             }
125         }
126
127         struct Candidate { int nx, ny, dist; };
128         std::vector<Candidate> closer;
129         std::vector<Candidate> farther;
130
131         auto consider = [&](int nx, int ny)
132         {
133             if (nx < 0 || ny < 0 || nx >= cols || ny >= rows)
134                 return;
135
136             if (visited[index(nx, ny)])
137                 return;

```

```

138
139         if (wouldCreateSquare(nx, ny, visited))
140             return;
141
142         const int dx[4] = { 1, -1, 0, 0 };
143         const int dy[4] = { 0, 0, 1, -1 };
144         for (int k = 0; k < 4; ++k)
145         {
146             int ax = nx + dx[k];
147             int ay = ny + dy[k];
148             if (ax < 0 || ay < 0 || ax >= cols || ay >= rows)
149                 continue;
150
151             if (!visited[index(ax, ay)])
152                 continue;
153
154             if (ax == x && ay == y)
155                 continue;
156
157             return;
158         }
159
160         int dNew = std::abs(nx - ex) + std::abs(ny - ey);
161         int dCurr = std::abs(x - ex) + std::abs(y - ey);
162
163         Candidate c{ nx, ny, dNew };
164         if (dNew <= dCurr)
165             closer.push_back(c);
166         else
167             farther.push_back(c);
168     };
169
170
171     consider(x + 1, y);
172     consider(x - 1, y);
173     consider(x, y + 1);
174     consider(x, y - 1);
175
176     if (closer.empty() && farther.empty())
177         break;
178
179     std::vector<Candidate>* pool = nullptr;
180     float r = prob01(rng);
181
182     if (!closer.empty() && (r < 0.7f || farther.empty()))
183         pool = &closer;
184     else
185         pool = &farther;
186
187     std::uniform_int_distribution<int> pick(0, (int)pool->size() - 1);
188     Candidate chosen = (*pool)[pick(rng)];

```

```

189
190         x = chosen.nx;
191         y = chosen.ny;
192         visited[index(x, y)] = 1;
193         tmpPath.push_back({ x, y });
194     }

```

Szum Perlina jako sposób rozmieszczania przeszkód

Po wyznaczeniu trasy uzupełniane są pozostałe pola planszy. Zamiast oznaczać przeszkody całkowicie losowo, zastosowano szum Perlina 2D, który daje wartości zmieniające się płynnie w przestrzeni. Dzięki temu przeszkody częściej tworzą skupiska, a mapa nie wygląda jak zbiór pojedynczych, przypadkowych kafelków.

Dla każdego pola poza trasą obliczana jest wartość szumu i porównywana z progiem. Po przekroczeniu progu kafelek jest oznaczany jako przeszkoda. Jednocześnie pola należące do drogi są chronione i nie mogą zostać nadpisane przeszkodą, co zachowuje przechodność mapy.

Implementacja wyznaczania trasy, wraz z obsługą prób generacji i reguł poprawności, znajduje się w metodzie `Grid::generateProceduralLevel` (Listing 2.1).

Listing 2.2. Funkcje pomocnicze do obliczania szumu Perlina 2D (fragment `Grid.cpp`).

```

1 namespace
2 {
3     inline float fade(float t)
4     {
5         return t * t * t * (t * (t * 6.f - 15.f) + 10.f);
6     }
7
8     inline float lerp(float a, float b, float t)
9     {
10         return a + t * (b - a);
11     }
12
13     inline std::uint32_t hash2D(int x, int y)
14     {
15         std::uint32_t h = static_cast<std::uint32_t>(x) * 374761393u +
16             static_cast<std::uint32_t>(y) * 668265263u;
17         h = (h ^ (h >> 13)) * 1274126177u;
18         return h ^ (h >> 16);
19     }
20
21     inline float grad2D(std::uint32_t h, float x, float y)
22     {
23         int hash = static_cast<int>(h & 7u);
24         float u = (hash < 4) ? x : y;
25         float v = (hash < 4) ? y : x;
26         return ((hash & 1) ? -u : u) + ((hash & 2) ? -v : v);
27     }
28
29     float perlinNoise2D(float x, float y)
30     {

```

```

31     int X = static_cast<int>(std::floor(x));
32     int Y = static_cast<int>(std::floor(y));
33
34     float xf = x - static_cast<float>(X);
35     float yf = y - static_cast<float>(Y);
36
37     float u = fade(xf);
38     float v = fade(yf);
39
40     std::uint32_t h00 = hash2D(X, Y);
41     std::uint32_t h10 = hash2D(X + 1, Y);
42     std::uint32_t h01 = hash2D(X, Y + 1);
43     std::uint32_t h11 = hash2D(X + 1, Y + 1);
44
45     float n00 = grad2D(h00, xf, yf);
46     float n10 = grad2D(h10, xf - 1.f, yf);
47     float n01 = grad2D(h01, xf, yf - 1.f);
48     float n11 = grad2D(h11, xf - 1.f, yf - 1.f);
49
50     float x1 = lerp(n00, n10, u);
51     float x2 = lerp(n01, n11, u);
52     float n = lerp(x1, x2, v);
53
54     return 0.5f * n + 0.5f;
55 }
56
57 float perlinFBM2D(float x, float y,
58     int octaves = 3, float lacunarity = 2.0f, float gain = 0.5f)
59 {
60     float amp = 1.0f;
61     float freq = 1.0f;
62     float sum = 0.0f;
63     float norm = 0.0f;
64
65     for (int i = 0; i < octaves; ++i)
66     {
67         sum += perlinNoise2D(x * freq, y * freq) * amp;
68         norm += amp;
69         amp *= gain;
70         freq *= lacunarity;
71     }
72     return (norm > 0.0f) ? (sum / norm) : 0.0f;
73 }
74
75 inline float perlin2D(float x, float y)
76 {
77     return perlinFBM2D(x, y, 3, 2.0f, 0.5f);
78 }
79 }

```

Właściwe rozmieszczenie przeszkód realizowane jest w końcowej części metody `Grid::generatePro` (Listing 2.3).

Listing 2.3. Etap 2: rozmieszczenie przeszkód na podstawie wartości szumu Perlina (fragment `Grid.cpp`).

```
1      float noiseScale = 0.15f;
2      float threshold = 0.55f;
3
4      for (int yy = 0; yy < rows; ++yy)
5      {
6          for (int xx = 0; xx < cols; ++xx)
7          {
8              if (types_[index(xx, yy)] != 0)
9                  continue;
10
11              float nx = xx * noiseScale + noiseShiftX;
12              float ny = yy * noiseScale + noiseShiftY;
13
14              float n = perlin2D(nx, ny);
15
16              if (n > threshold)
17                  types_[index(xx, yy)] = 2;
18          }
19      }
20
21      if (uploadTexture)
22          uploadAll();
23
24      return;
25  }
```

2.4.2. Parametry wież i przeciwników

Poniższe zestawienia pokazują wartości bazowe jednostek użytych w rozgrywce. Parametry wież (Tabela 2.4) stanowią punkt wyjścia dla logiki gry, na przykład przy przeliczaniu statystyk po ulepszeniu wieży.

Tabela 2.4. Parametry bazowe wież dostępnych w grze.

Nazwa	Opis i parametry (wartości bazowe)	Grafika
<i>Basic</i>	Uniwersalna wieża o zbalansowanych parametrach. Obrażenia: 10,0; zasięg: 2,5; szybkostrzelność: 1,0; prędkość pocisku: 4,0; koszt: 100.	
<i>Slow</i>	Wieża o niższych obrażeniach, zapewniająca spowolnienie trafionych przeciwników. Obrażenia: 4,0; zasięg: 2,0; szybkostrzelność: 0,6; prędkość pocisku: 4,0; koszt: 120. Efekt spowolnienia: współczynnik 0,5 przez 1,2 s.	
<i>Fast</i>	Szybkostrzelna wieża o niskich obrażeniach i wysokiej prędkości pocisku. Obrażenia: 2,0; zasięg: 1,8; szybkostrzelność: 3,0; prędkość pocisku: 6,0; koszt: 90.	

W przypadku ulepszeń wież przyjęto koszt: $\text{koszt_ulepszenia} = \text{koszt_bazowy} \cdot \text{poziom}$. Parametry wieży są modyfikowane mnożnikami zależnymi od poziomu: obrażenia $1,0 + 0,6(\ell - 1)$, zasięg $1,0 + 0,2(\ell - 1)$, szybkostrzelność $1,0 + 0,3(\ell - 1)$. Parametry bazowe przeciwników, wykorzystywane w dalszej części logiki rozgrywki, ujęto w tabeli 2.5.

Tabela 2.5. Parametry bazowe przeciwników występujących w grze.

Nazwa	Opis i parametry (wartości bazowe)	Grafika
<i>Basic enemy</i>	Przeciwnik podstawowy o zbalansowanych parametrach. HP: 30,0; prędkość: 2,0; nagroda: 15 złota; obrażenia gracza: 1.	
<i>Fast enemy</i>	Szybki przeciwnik o niższej wytrzymałości. HP: 18,0; prędkość: 3,2; nagroda: 12 złota; obrażenia gracza: 1.	
<i>Tank enemy</i>	Wolny, bardzo wytrzymały przeciwnik. HP: 80,0; prędkość: 1,2; nagroda: 30 złota; obrażenia gracza: 1.	

Powiązanie z wariantami tekstur. Po ustaleniu typów pól (tło/droga/przeszkoda) wizualna różnorodność kafelków jest dodatkowo zwiększana przez losowy dobór wariantów tekstur w shaderze na podstawie parametru `uSeed`. Mechanizm ten został opisany w podsekcji dotyczącej losowych wariantów kafelków.

2.4.3. Przekazanie danych planszy do shaderów i renderowanie siatki

W implementacji plansza gry jest renderowana jako siatka kafelków w przestrzeni znormalizowanych współrzędnych urządzenia (NDC), tj. w zakresie $[-1, 1]$ dla osi x oraz y . Każdy kafelek jest geometrycznie tym samym prostokątem złożonym z dwóch trójkątów, natomiast jego położenie na ekranie wynika z identyfikatora instancji (`gl_InstanceID`) w shaderze wierzchołków.

Shader wierzchołków siatki (`gridVS`)

Shader wierzchołków siatki (`gridVS`) wyznacza położenie instancji kafelków na podstawie identyfikatora `gl_InstanceID` (Listing 2.4).

Listing 2.4. Shader wierzchołków siatki (GLSL)

```

1  #version 330 core
2  layout (location = 0) in vec2 aPos;\
3  uniform ivec2 uGrid;
4  uniform vec2 uTileSize;
5  uniform vec2 uGap;
6  uniform vec2 uOffset;
7  flat out ivec2 vCell;
8  out vec2 vUV;
9  void main(){
10     int col = gl_InstanceID % uGrid.x;
11     int row = gl_InstanceID / uGrid.x;
12     vCell = ivec2(col,row);
13
14     vec2 gridCenter = (vec2(uGrid) - 1.0) * 0.5;
15     vec2 offset = (vec2(col,row) - gridCenter) * (uTileSize + uGap) + uOffset;
16     gl_Position = vec4(aPos * uTileSize + offset, 0.0, 1.0);
17
18     vUV = aPos + vec2(0.5);
19 }

```

W celu ograniczenia liczby wywołań rysowania zastosowano instancjonowanie (*instanced rendering*). Jedno wywołanie rysowania generuje $cols \cdot rows$ instancji kafelków, odpowiadających wszystkim komórkom siatki. W shaderze wierzchołków dla siatki wyznaczane są współrzędne komórki:

— $col = gl_InstanceID \bmod cols$,

— $row = \lfloor gl_InstanceID / cols \rfloor$,

a następnie obliczane jest przesunięcie kafelka na podstawie rozmiaru kafelka, odstępów oraz wektora przesunięcia całej siatki. Pozwala to na dynamiczne dopasowanie planszy do wydzielonego obszaru okna aplikacji.

Wywołanie rysowania instancjonowanego

Wywołanie rysowania instancjonowanego, ograniczające liczbę operacji renderowania do jednego wywołania dla całej siatki, pokazano w listingu 2.5.

Listing 2.5. Fragment renderowania siatki z użyciem instancjonowania

```

1  app.shGrid->use();
2
3  app.grid->bindTextureUnit(0);
4
5  glActiveTexture(GL_TEXTURE1);
6  glBindTexture(GL_TEXTURE_2D, app.grassTex);
7
8  glActiveTexture(GL_TEXTURE2);
9  glBindTexture(GL_TEXTURE_2D, app.roadTex);
10
11 glActiveTexture(GL_TEXTURE3);
12 glBindTexture(GL_TEXTURE_2D, app.startGoalTex);
13
14 glActiveTexture(GL_TEXTURE4);

```



```

15 glBindTexture(GL_TEXTURE_2D, app.obstacleTex);
16
17 glBindVertexArray(app.VAO);
18 glDrawElementsInstanced(GL_TRIANGLES, 6, GL_UNSIGNED_INT, 0, gp.cols * gp.rows);

```

2.4.4. Tekstura typów pól (uTypes)

Typy pól planszy są przechowywane w obiekcie siatki jako tablica wartości całkowitych, a następnie przesyłane na GPU w postaci jednowymiarowego kanału tekstury 2D w formacie 8-bitowym (GL_R8). Tekstura ta jest próbkowana w shaderze fragmentów funkcją `texelFetch`, co zapewnia bezpośredni dostęp do wartości typu pola dla wskazanej komórki siatki.

Odczytana wartość jest interpretowana jako typ pola (tło, droga, przeszkoda), a następnie wykorzystywana do wyboru odpowiedniego fragmentu atlasu tekstur. Takie podejście upraszcza logikę renderowania: geometria jest stała, natomiast wygląd kafelka wynika z danych przechowywanych w teksturze typów.

Shader fragmentów siatki (gridFS)

Shader fragmentów siatki (`gridFS`) odpowiada za interpretację typu pola oraz wybór odpowiedniego fragmentu atlasu tekstur (Listing 2.6).

Listing 2.6. Shader fragmentów siatki (GLSL)

```

1  #version 330 core
2  flat in ivec2 vCell;
3  in vec2 vUV;
4
5  uniform sampler2D uTypes;
6  uniform sampler2D uGrassAtlas;
7  uniform sampler2D uRoadAtlas;
8  uniform sampler2D uStartGoalAtlas;
9  uniform vec3 uColorRoad;
10 uniform vec3 uColorBlock;
11 uniform float uSeed;
12 uniform ivec2 uGrid;
13 uniform ivec2 uStartCell;
14 uniform ivec2 uGoalCell;
15 uniform sampler2D uObstacleAtlas;
16
17 out vec4 FragColor;
18
19 float hash2(vec2 p)
20 {
21     return fract(sin(dot(p, vec2(127.1, 311.7)))) * 43758.5453);
22 }
23
24 int getCellType(ivec2 c)
25 {
26     if (c.x < 0 || c.y < 0 || c.x >= uGrid.x || c.y >= uGrid.y)
27         return 0;
28     float t = texelFetch(uTypes, c, 0).r;

```

```

29     return int(round(t * 255.0));
30 }
31
32 vec4 sampleGrass()
33 {
34     const float tilesX = 4.0;
35     const float tilesY = 2.0;
36
37     float r = hash2(vec2(vCell) + vec2(uSeed, uSeed * 1.37));
38     int idx = int(floor(r * 8.0)) % 8;
39     int ix = idx % int(tilesX);
40     int iy = idx / int(tilesX);
41
42     vec2 tileScale = vec2(1.0 / tilesX, 1.0 / tilesY);
43     vec2 baseUV = vec2(float(ix), float(iy)) * tileScale;
44
45     vec2 local = vec2(vUV.x, 1.0 - vUV.y);
46
47     vec2 atlasUV = baseUV + local * tileScale;
48     vec3 col = texture(uGrassAtlas, atlasUV).rgb;
49     return vec4(col, 1.0);
50 }
51
52 vec4 sampleRoad()
53 {
54     int up    = getCellType(vCell + ivec2(0, 1));
55     int down  = getCellType(vCell + ivec2(0, -1));
56     int left  = getCellType(vCell + ivec2(-1, 0));
57     int right = getCellType(vCell + ivec2(1, 0));
58
59     bool bUp    = (up == 1);
60     bool bDown  = (down == 1);
61     bool bLeft  = (left == 1);
62     bool bRight = (right == 1);
63
64     int neighbors = int(bUp) + int(bDown) + int(bLeft) + int(bRight);
65
66     const int IDX_H  = 0;
67     const int IDX_DL = 1;
68     const int IDX_UR = 2;
69     const int IDX_LU = 3;
70     const int IDX_V  = 4;
71     const int IDX_RD = 5;
72
73     int roadIdx = IDX_H;
74
75     if (neighbors <= 1)
76     {
77         if (bLeft || bRight)
78             roadIdx = IDX_H;
79         else

```

```

80         roadIdx = IDX_V;
81     }
82     else if (neighbors == 2)
83     {
84         bool horizontal = (bLeft && bRight);
85         bool vertical    = (bUp    && bDown);
86         bool onlyLR      = (bLeft || bRight) && !(bUp || bDown);
87         bool onlyUD      = (bUp    || bDown) && !(bLeft || bRight);
88
89         if (horizontal || onlyLR)
90         {
91             roadIdx = IDX_H;
92         }
93         else if (vertical || onlyUD)
94         {
95             roadIdx = IDX_V;
96         }
97         else
98         {
99             if (bUp && bRight)      roadIdx = IDX_UR;
100             else if (bRight && bDown) roadIdx = IDX_RD;
101             else if (bDown && bLeft) roadIdx = IDX_DL;
102             else if (bLeft && bUp)   roadIdx = IDX_LU;
103         }
104     }
105     else
106     {
107         roadIdx = IDX_V;
108     }
109
110     const float tilesX = 3.0;
111     const float tilesY = 2.0;
112
113     int ix = roadIdx % int(tilesX);
114     int iy = roadIdx / int(tilesX);
115
116     vec2 tileScale = vec2(1.0 / tilesX, 1.0 / tilesY);
117     vec2 baseUV = vec2(float(ix), float(iy)) * tileScale;
118
119     vec2 local = vec2(vUV.x, 1.0 - vUV.y);
120     vec2 atlasUV = baseUV + local * tileScale;
121     vec3 col = texture(uRoadAtlas, atlasUV).rgb;
122     return vec4(col, 1.0);
123 }
124 vec4 sampleObstacle()
125 {
126     const float tilesX = 2.0;
127     const float tilesY = 2.0;
128
129     float r = hash2(vec2(vCell) + vec2(uSeed * 2.17, uSeed * 3.31));
130     int idx = int(floor(r * 4.0)) % 4;

```

```

131
132     int ix = idx % int(tilesX);
133     int iy = idx / int(tilesX);
134
135     vec2 tileScale = vec2(1.0 / tilesX, 1.0 / tilesY);
136     vec2 baseUV = vec2(float(ix), float(iy)) * tileScale;
137
138     vec2 local = vec2(vUV.x, 1.0 - vUV.y);
139     vec2 atlasUV = baseUV + local * tileScale;
140
141     vec3 col = texture(uObstacleAtlas, atlasUV).rgb;
142     return vec4(col, 1.0);
143 }
144 vec4 sampleStartGoal(bool isStart)
145 {
146     const float tilesX = 1.0;
147     const float tilesY = 2.0;
148
149     int idx = isStart ? 0 : 1;
150     int ix = 0;
151     int iy = idx;
152
153     vec2 tileScale = vec2(1.0 / tilesX, 1.0 / tilesY);
154     vec2 baseUV = vec2(float(ix), float(iy)) * tileScale;
155
156     vec2 local = vec2(vUV.x, 1.0 - vUV.y);
157     vec2 atlasUV = baseUV + local * tileScale;
158     return texture(uStartGoalAtlas, atlasUV);
159 }
160
161
162
163 void main()
164 {
165     int ty = getCellType(vCell);
166
167     bool isStart = all(equal(vCell, uStartCell));
168     bool isGoal  = all(equal(vCell, uGoalCell));
169
170     if (ty == 1)
171     {
172         if (isStart)
173         {
174             FragColor = sampleStartGoal(true);
175             return;
176         }
177         if (isGoal)
178         {
179             FragColor = sampleStartGoal(false);
180             return;
181         }

```

```

182
183     FragColor = sampleRoad();
184     return;
185 }
186 if (ty == 2)
187 {
188     FragColor = sampleObstacle();
189     return;
190 }
191
192 FragColor = sampleGrass();
193 }

```

2.4.5. Wyróżnienie pola startowego i docelowego (uStartCell, uGoalCell)

Po wygenerowaniu ścieżki głównej przez moduł generowania proceduralnego wyznaczone są współrzędne pola startowego oraz docelowego jako odpowiednio pierwszy i ostatni element listy współrzędnych ścieżki. Współrzędne te są przekazywane do shadera w postaci uniformów `uStartCell` oraz `uGoalCell`.

W shaderze fragmentów, dla pól należących do drogi, wykonywane jest sprawdzenie, czy aktualnie renderowana komórka odpowiada polu startowemu lub docelowemu. Jeżeli warunek jest spełniony, zamiast standardowego kafelka drogi używany jest dedykowany kafel z atlasu `uStartGoalAtlas`. Należy podkreślić, że pola startowe i docelowe nie są kodowane jako odrębne typy planszy - pozostają elementami drogi, a wyróżnienie ma charakter wyłącznie wizualny.

2.4.6. Losowe warianty kafelków (uSeed)

Aby ograniczyć powtarzalność grafiki, wprowadzono wariantowanie kafelków tła oraz przeszkód. W shaderze fragmentów wykorzystywana jest deterministyczna funkcja mieszająca, która na podstawie współrzędnych komórki oraz parametru `uSeed` wybiera indeks kafelka w atlasie. Parametr `uSeed` jest losowany przy rozpoczęciu gry oraz przy generowaniu nowej mapy.

Rozwiązanie to pozwala uzyskać zróżnicowany wygląd planszy bez konieczności modyfikacji geometrii ani przechowywania dodatkowych danych w strukturze planszy. Zmiana wartości `uSeed` prowadzi do innego rozkładu wariantów kafelków, przy zachowaniu tej samej logiki typów pól.

2.4.7. Odświeżenie mapy podczas rozgrywki

W aplikacji przewidziano możliwość wygenerowania nowej mapy (np. z poziomu menu). W takim przypadku ponownie wywoływana jest procedura generowania planszy, po czym aktualizowane są:

- uniformy `uStartCell` oraz `uGoalCell` na podstawie nowej ścieżki,
- parametr `uSeed` odpowiadający za warianty kafelków.

Dzięki temu logika gry i renderowanie pozostają spójne po każdej zmianie mapy, a aktualizacja ogranicza się do przeliczenia danych planszy oraz ustawienia kilku uniformów.

2.4.8. Struktura aplikacji i pętla główna gry

Aplikacja została zrealizowana jako program okienkowy z pętlą główną czasu rzeczywistego. Inicjalizacja obejmuje przygotowanie biblioteki GLFW, utworzenie okna, skonfigurowanie

kontekstu OpenGL oraz załadowanie wskaźników funkcji API z użyciem biblioteki GLAD. Następnie inicjalizowany jest interfejs użytkownika w bibliotece ImGui oraz tworzona jest geometria bazowa (prostokąt złożony z dwóch trójkątów), wykorzystywana do renderowania kafelków planszy i obiektów gry.

2.4.9. Inicjalizacja zasobów renderowania

W fazie uruchomieniowej tworzone są obiekty shaderów wykorzystywane do renderowania poszczególnych elementów sceny, w tym siatki mapy, przeciwników, wież, pocisków, podświetlenia zaznaczenia oraz elementów panelu bocznego. Dodatkowo ładowane są tekstury z plików graficznych przy użyciu biblioteki `stb_image` i wiązane do ustalonych jednostek tekstur. Ujednolicenie jednostek tekstur umożliwia prostą konfigurację samplerów w shaderach poprzez ustawienie stałych indeksów.

Abstrakcja shaderów w C++ (klasa Shader)

Interfejs klasy `Shader` oraz jej implementacja upraszczają kompilację i linkowanie shaderów, a także ustawianie parametrów *uniform* (Listing 2.7 oraz Listing 2.8).

Listing 2.7. Klasa `Shader` - plik nagłówkowy

```
1 #pragma once
2 #include <glad/glad.h>
3 #include <string>
4
5 class Shader {
6 public:
7     Shader(const char* vs, const char* fs);
8     ~Shader();
9     void use() const;
10    GLint loc(const char* name) const;
11    GLuint id() const { return program_; }
12 private:
13    GLuint program_{ 0 };
14    static GLuint compile(GLenum type, const char* src);
15 };
```

Listing 2.8. Klasa `Shader` - implementacja

```
1 #include "Shader.h"
2 #include <vector>
3 #include <cstring>
4 #include <iostream>
5
6 GLuint Shader::compile(GLenum type, const char* src) {
7     GLuint s = glCreateShader(type);
8     GLint len = (GLint)std::strlen(src);
9     glShaderSource(s, 1, &src, &len);
10    glCompileShader(s);
11    GLint ok = 0; glGetShaderiv(s, GL_COMPILE_STATUS, &ok);
12    if (!ok) { GLint L = 0; glGetShaderiv(s, GL_INFO_LOG_LENGTH, &L);
13        std::string log(L, '\0');
```

```

14         glGetShaderInfoLog(s, L, nullptr, (GLchar*)log.data());
15         std::cerr << log << '\n';
16         std::exit(1); }
17     return s;
18 }
19
20 Shader::Shader(const char* vs, const char* fs) {
21     GLuint v = compile(GL_VERTEX_SHADER, vs);
22     GLuint f = compile(GL_FRAGMENT_SHADER, fs);
23     program_ = glCreateProgram();
24     glAttachShader(program_, v);
25     glAttachShader(program_, f);
26     glLinkProgram(program_);
27     GLint ok = 0; glGetProgramiv(program_, GL_LINK_STATUS, &ok);
28     if (!ok) { GLint L = 0; glGetProgramiv(program_, GL_INFO_LOG_LENGTH, &L);
29         std::string log(L, '\0');
30         glGetProgramInfoLog(program_, L, nullptr, (GLchar*)log.data());
31         std::cerr << log << '\n'; std::exit(1); }
32     glDeleteShader(v);
33     glDeleteShader(f);
34 }
35
36 Shader::~Shader() {
37     if (program_) glDeleteProgram(program_);
38 }
39
40 void Shader::use() const {
41     glUseProgram(program_);
42 }
43
44 GLint Shader::loc(const char* name) const {
45     return glGetUniformLocation(program_, name);
46 }

```

2.4.10. Konfiguracja siatki i dopasowanie do obszaru okna

Plansza gry jest definiowana parametrami liczby kolumn i wierszy oraz bazowego rozmiaru kafelka. Następnie obliczany jest współczynnik skali zapewniający dopasowanie całej siatki do wydzielonego obszaru ekranu przeznaczonego na mapę (z uwzględnieniem marginesów oraz podziału okna na część gry i panel sterowania). Wektor przesunięcia siatki jest ustawiany tak, aby mapa była wyśrodkowana w swoim regionie.

2.4.11. Obsługa wejścia i wybór komórki

W aplikacji zdefiniowano obsługę kliknięcia lewym przyciskiem myszy. Współrzędne kursora są przeliczane na układ NDC, a następnie przekazywane do funkcji siatki, która wyznacza, czy kliknięcie mieści się w granicach konkretnego kafelka. Jeżeli trafiono w kafelek, zapisywane są jego współrzędne siatkowe oraz typ pola, co stanowi podstawę do dalszych działań, takich jak budowa wieży lub prezentacja informacji w panelu.

2.4.12. Pętla główna i aktualizacja stanu gry

Pętla główna jest wykonywana do momentu zamknięcia okna. W każdej iteracji obliczany jest krok czasowy Δt na podstawie aktualnego czasu i czasu poprzedniej klatki. Następnie wykonywana jest obsługa zdarzeń wejściowych oraz aktualizacja logiki gry.

Aktualizacja mechanik rozgrywki jest realizowana warunkowo: system fal przeciwników oraz system pocisków są aktualizowane wyłącznie wtedy, gdy gracz znajduje się w trybie rozgrywki, a gra nie jest spauzowana ani zakończona. W trakcie aktualizacji weryfikowane są warunki porażki (spadek punktów życia do zera) oraz warunki zwycięstwa (zakończenie wszystkich fal i brak aktywnych przeciwników).

2.4.13. Renderowanie sceny

W każdej klatce wykonywane jest czyszczenie bufora koloru, a następnie renderowanie elementów gry. Najpierw rysowana jest mapa w postaci siatki kafelków z wykorzystaniem instancjonowania. Następnie, jeśli gracz zaznaczył komórkę, rysowane jest jej podświetlenie. Kolejne etapy obejmują renderowanie tła panelu bocznego, wizualizację zasięgu zaznaczonej wieży, rysowanie przeciwników, pocisków oraz wież. Na końcu renderowana jest warstwa interfejsu ImGui i wykonywana jest zamiana buforów ekranu.

2.4.14. Zwalnianie zasobów

Po zakończeniu działania aplikacji zwalniane są zasoby OpenGL (bufory i obiekty VAO) oraz zamykane są konteksty ImGui. Na końcu niszczone jest okno GLFW i wykonywana jest finalizacja biblioteki.

2.5. Instrukcja użytkownika

Niniejsza sekcja opisuje obsługę gry z perspektywy użytkownika. Wszystkie napisy w interfejsie (nazwy przycisków, komunikaty) są wyświetlane w języku angielskim i w tej formie są przywoływane w opisie.

2.5.1. Sterowanie (podstawy)

Sterowanie odbywa się za pomocą myszy:

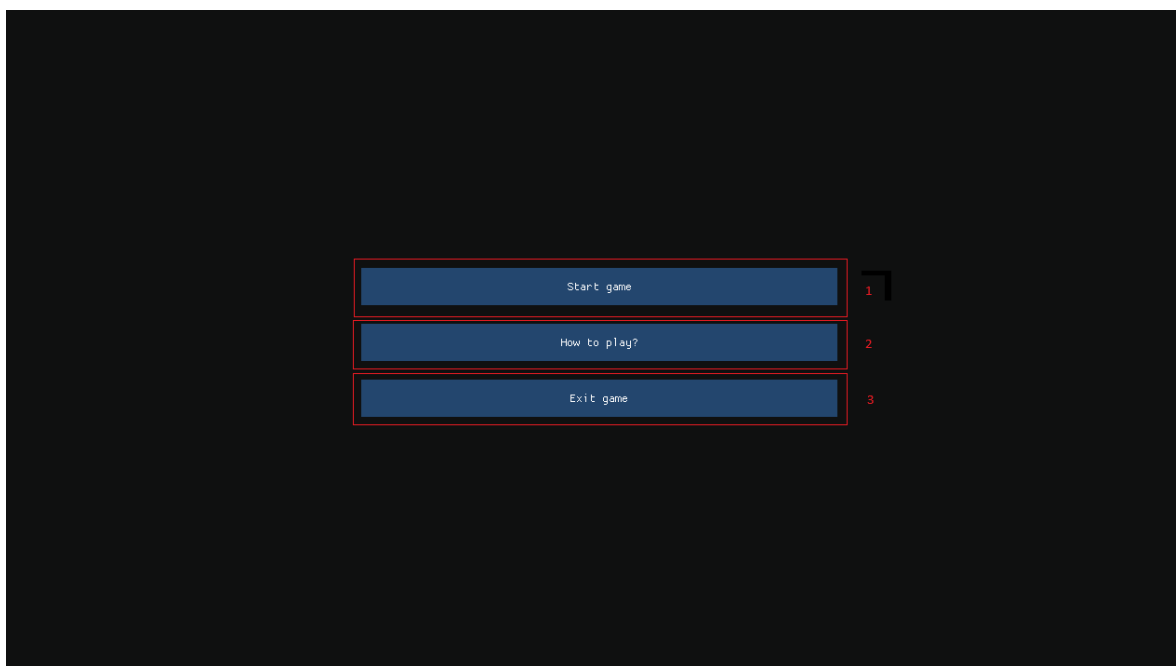
- kliknięcie lewym przyciskiem myszy na mapie służy do wyboru kafelka,
- kliknięcie na kafelku z wieżą służy do zaznaczenia wieży i wyświetlenia jej parametrów,
- przyciski w panelu po prawej stronie uruchamiają falę, otwierają menu oraz pozwalają budować, ulepszać i sprzedawać wieże.

2.5.2. Menu główne

Po uruchomieniu aplikacji wyświetla się ekran menu głównego z trzema przyciskami: *Start game*, *How to play?*, *Exit game*.

- *Start game* - rozpoczyna nową rozgrywkę (generowana jest mapa i inicjalizowany stan gry),
- *How to play?* - otwiera okno z krótką instrukcją zasad,
- *Exit game* - zamyka aplikację.

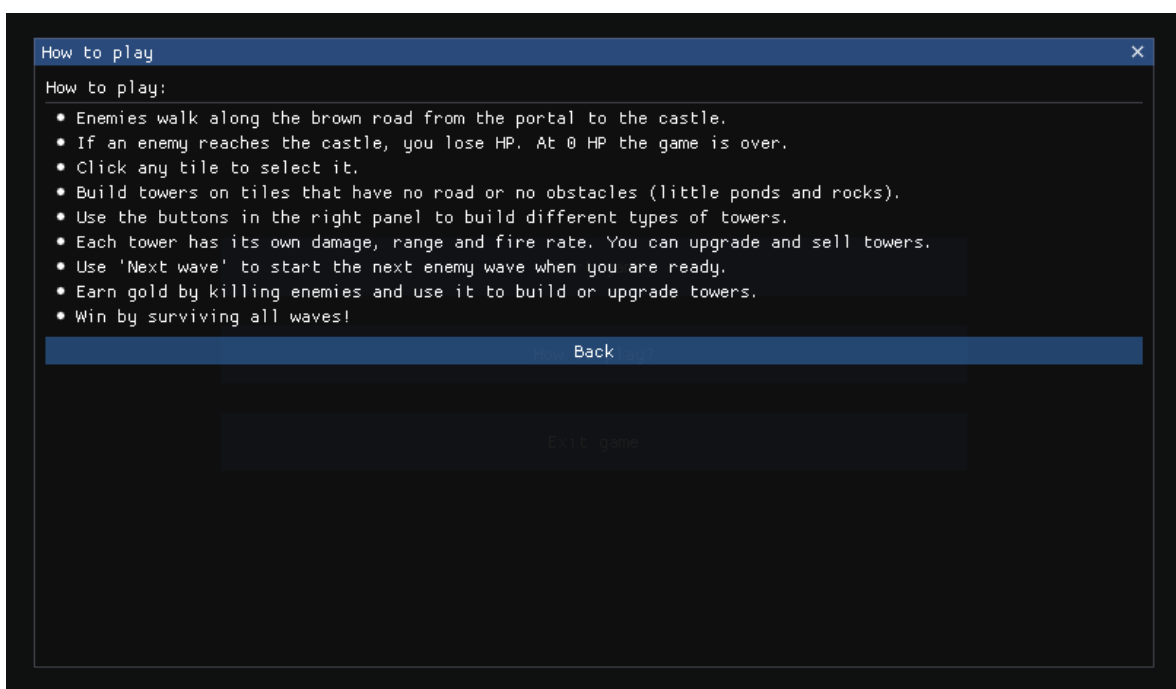
Układ menu głównego z trzema przyciskami odpowiada opisanej strukturze (Rys. 2.4).



Rysunek 2.4. Zrzut ekranu: menu główne gry.

2.5.3. Okno instrukcji *How to play?*

Po wybraniu *How to play?* wyświetlane jest okno z punktową instrukcją (Rys. 2.5). Powrót do menu głównego odbywa się przyciskiem *Back*.



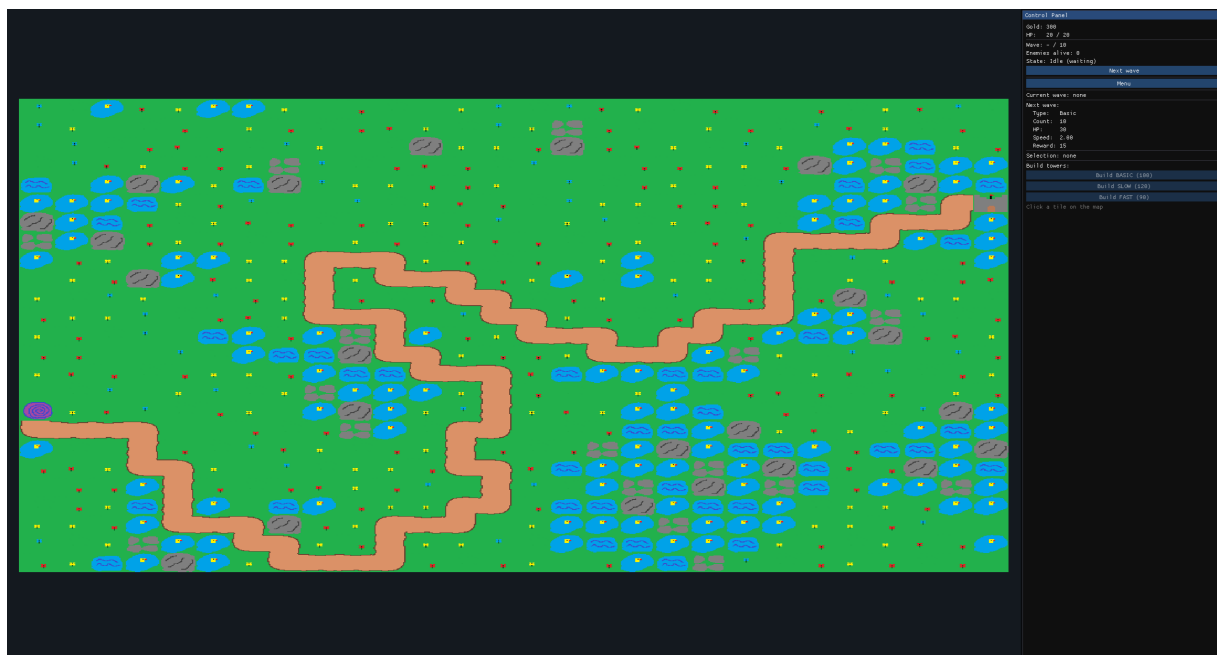
Rysunek 2.5. Zrzut ekranu: okno *How to play?* dostępne z menu głównego.

2.5.4. Start rozgrywki i podział ekranu

Po rozpoczęciu gry ekran jest podzielony na dwie strefy:

- obszar mapy (po lewej) - siatka kafelków z drogą przeciwników,
- panel sterowania (po prawej) - informacje o stanie gry i przyciski sterujące.

Podział ekranu na obszar mapy oraz panel sterowania jest widoczny po rozpoczęciu rozgrywki (Rys. 2.6).



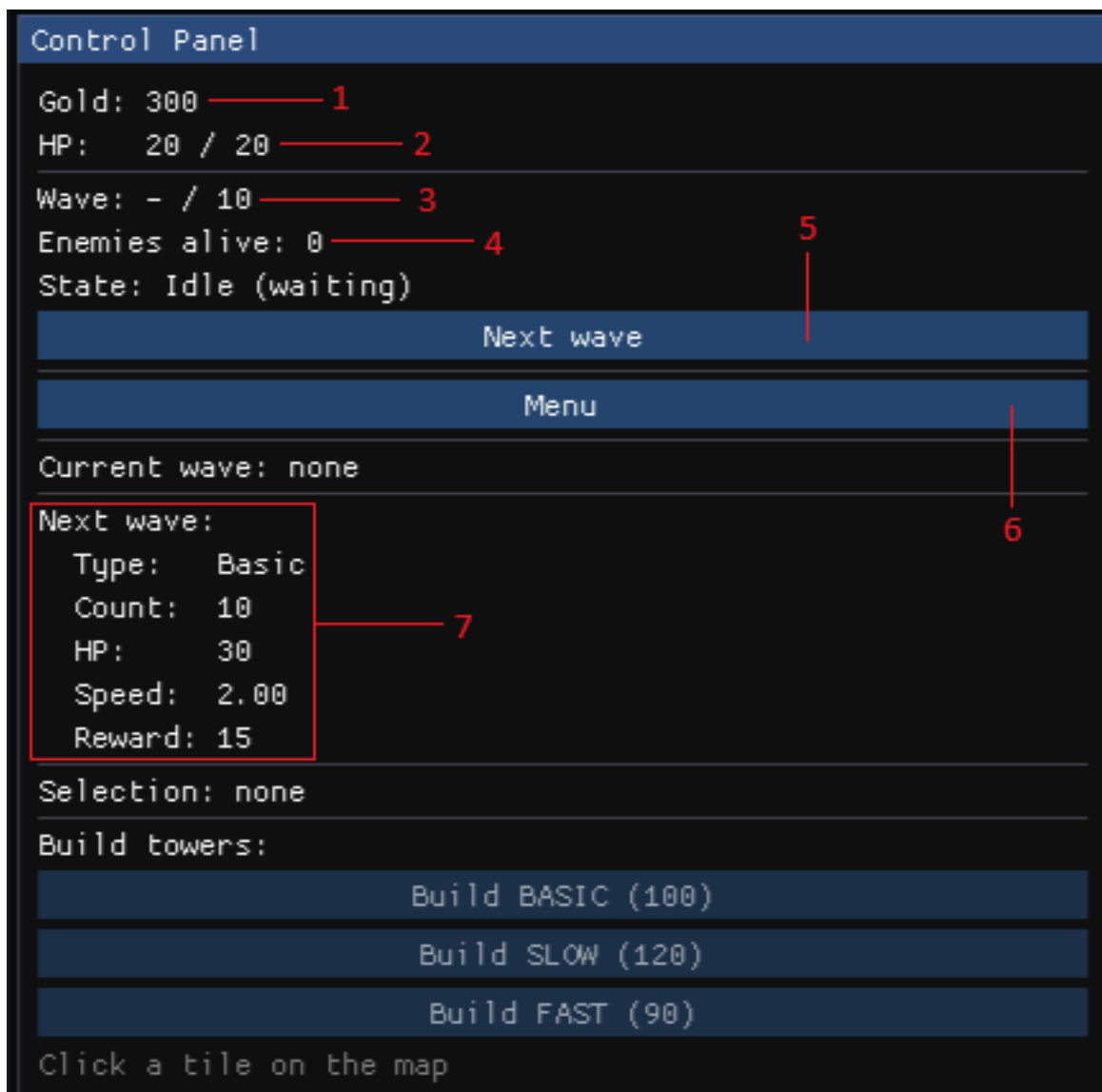
Rysunek 2.6. Zrzut ekranu: widok rozgrywki po rozpoczęciu gry.

2.5.5. Panel sterowania bez zaznaczenia

Gdy nie jest zaznaczony ani kafelek, ani wieża, panel sterowania prezentuje podstawowe informacje o stanie gry oraz udostępnia przyciski *Next wave* i *Menu*. Na ekranie panelu wyróżniono elementy oznaczeniami:

- (1) *Gold* - ilość złota dostępna do budowy i ulepszeń,
- (2) *HP* - punkty życia gracza,
- (3) *Wave* - numer aktualnej fali,
- (4) *Enemies alive* - liczba żywych przeciwników,
- (5) *Next wave* - uruchomienie kolejnej fali przeciwników,
- (6) *Menu* - otwarcie menu pauzy,
- (7) statystyki przeciwników występujących w następnej fali.

W tym stanie przyciski budowania wież pozostają nieaktywne do momentu wybrania kafelka możliwego do zabudowy. Układ panelu sterowania w stanie braku zaznaczenia odzwierciedla wymienione elementy interfejsu (Rys. 2.7).



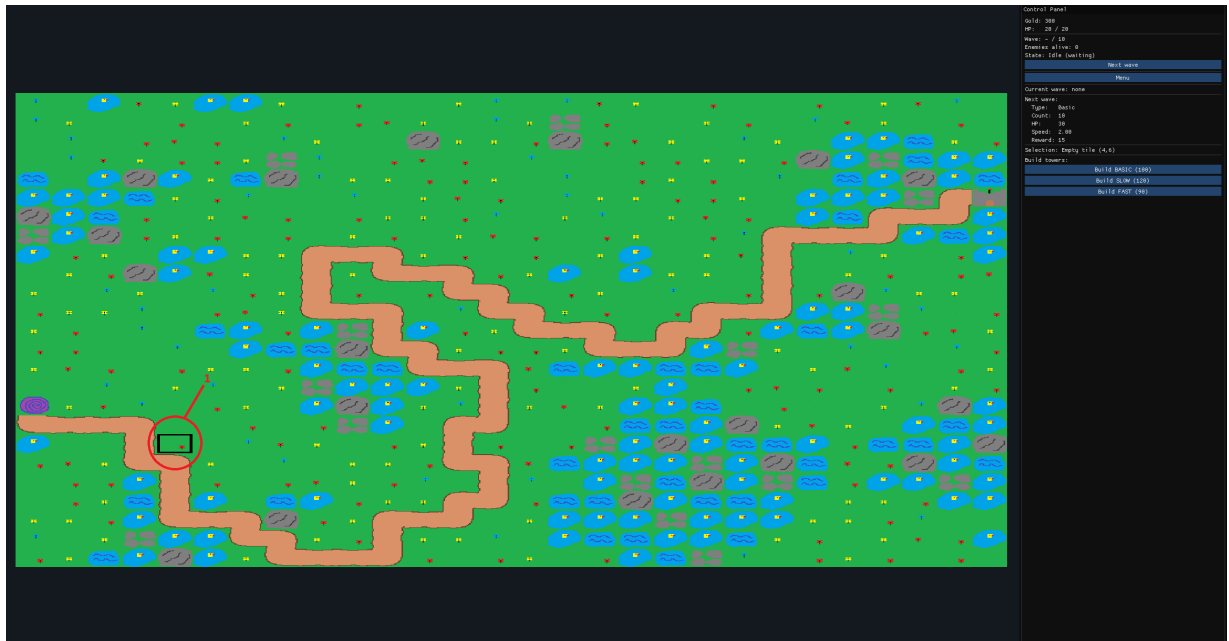
Rysunek 2.7. Zrzut ekranu: panel sterowania bez zaznaczenia kafelka i wieży.

2.5.6. Wybór kafelka na mapie

Kliknięcie w obszar mapy wybiera kafelek. Zaznaczenie jest wizualnie podświetlane, a w panelu pojawia się informacja o aktualnym wyborze. Budowanie wież jest możliwe wyłącznie na kafelkach, które:

- nie należą do drogi przeciwników,
- nie są przeszkodą,
- nie są już zajęte przez inną wieżę.

Przykład zaznaczenia kafelka na mapie, wraz z podświetleniem wyboru, pokazuje rysunek 2.8.



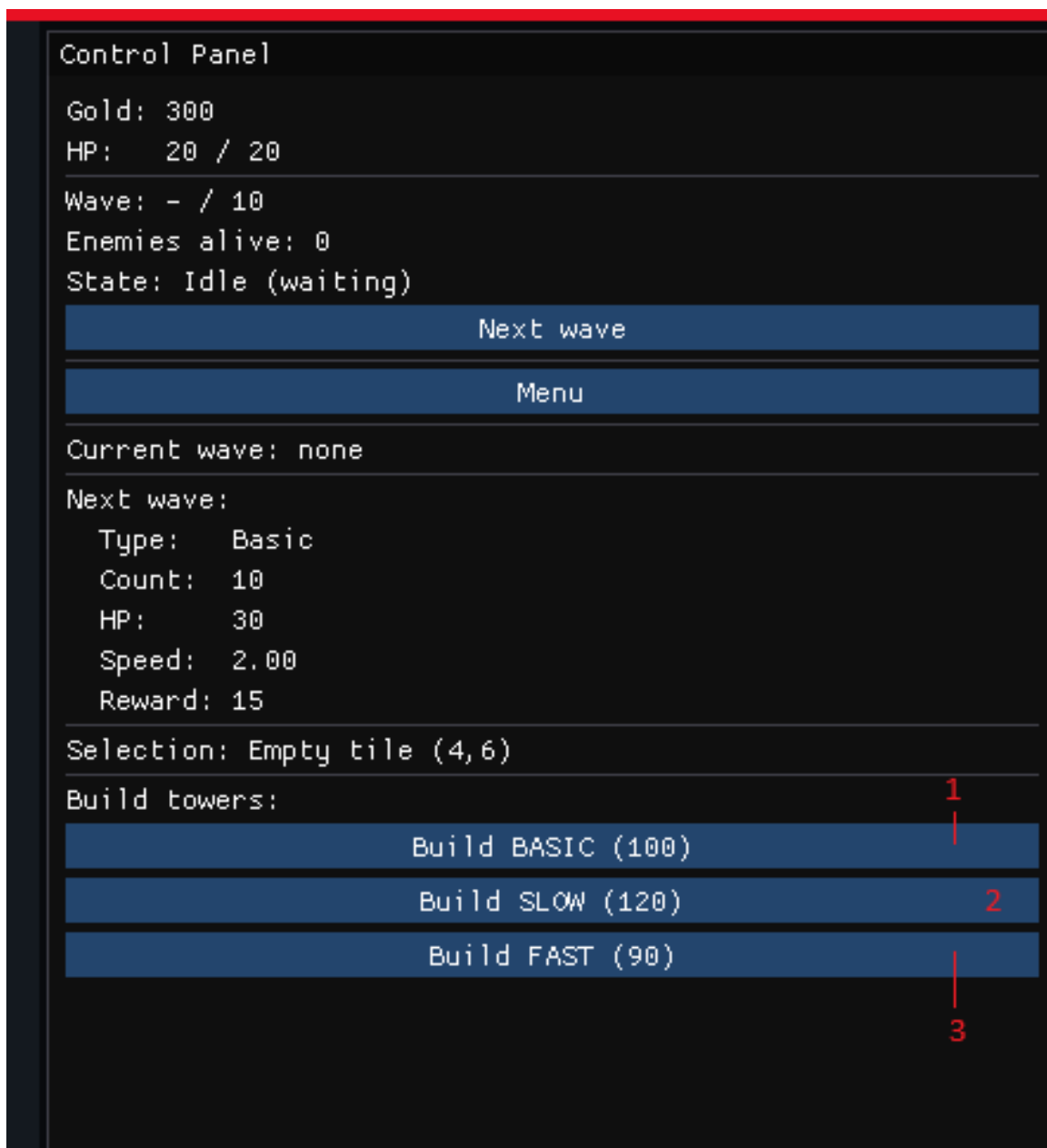
Rysunek 2.8. Zrzut ekranu: zaznaczenie kafelka na mapie.

2.5.7. Budowa wieży

Po zaznaczeniu kafelka, na którym można budować, w panelu aktywują się przyciski budowy:

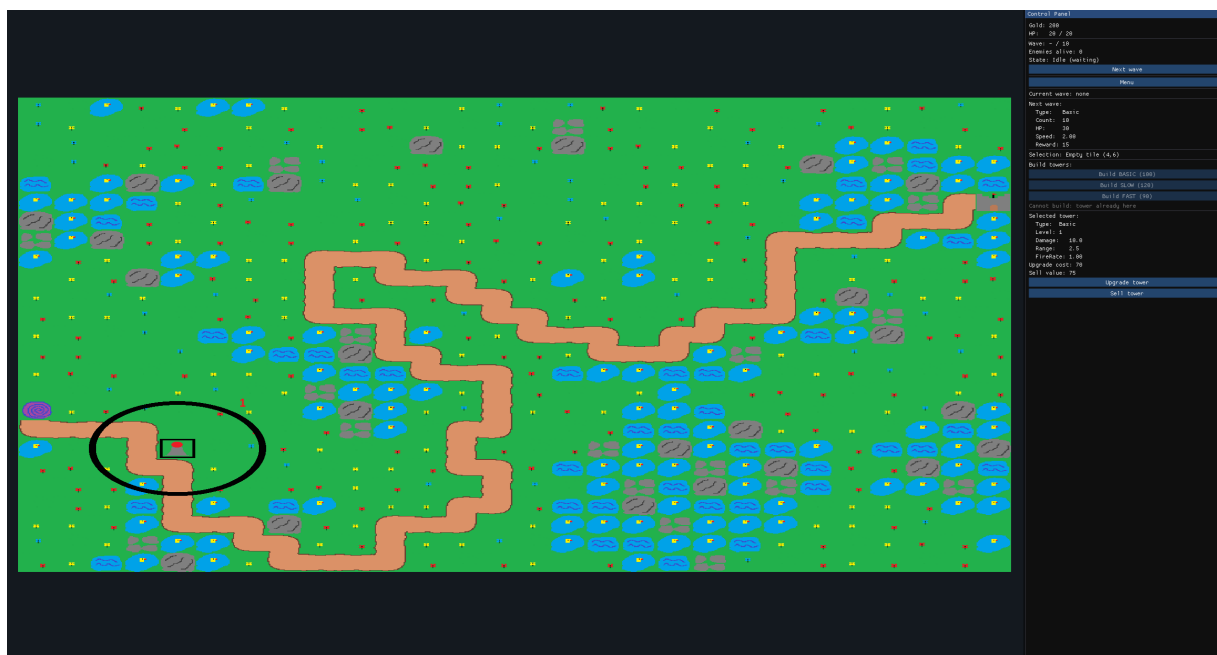
- (1) *Build BASIC* - budowa wieży *Basic*,
- (2) *Build SLOW* - budowa wieży *Slow*,
- (3) *Build FAST* - budowa wieży *Fast*.

Koszt budowy jest widoczny w nazwie przycisku. Jeżeli gracz nie posiada wystarczającej ilości złota, odpowiedni przycisk pozostaje nieaktywny. Po zaznaczeniu kafelka możliwego do zabudowy w panelu aktywują się przyciski budowy, wraz z widocznymi kosztami (Rys. 2.9).



Rysunek 2.9. Zrzut ekranu: panel budowy wież po zaznaczeniu kafelka możliwego do zabudowy.

Po zbudowaniu wieży kafelek staje się zajęty. Próba budowy na tym samym kafelku powoduje wyświetlenie komunikatu blokującego (kafelek jest już zajęty). Przykład zbudowanej wieży wraz z komunikatem blokującym przedstawiono na rysunku 2.10.



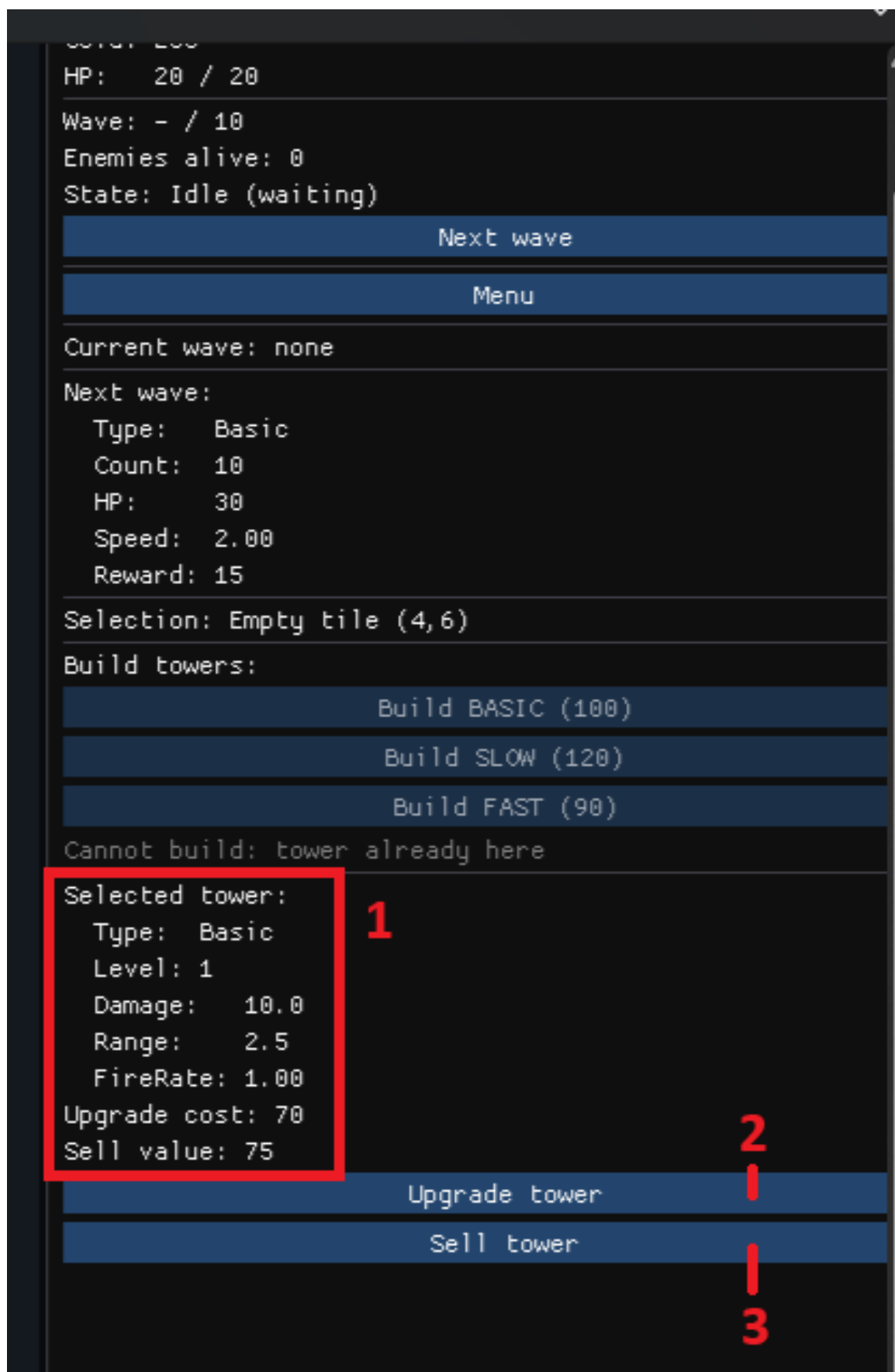
Rysunek 2.10. Zrzut ekranu: zbudowana wieża i komunikat blokujący ponowną budowę na tym samym kafelku.

2.5.8. Zaznaczenie wieży, ulepszenie i sprzedaż

Kliknięcie w kafelek z wieżą zaznacza wieżę i powoduje wyświetlenie jej parametrów w panelu sterowania. W panelu dostępne są:

- (1) blok informacji o wieży (typ, poziom oraz statystyki),
- (2) *Upgrade tower* - ulepszenie wieży (zwiększa poziom i parametry, jeśli dostępne jest złoto),
- (3) *Sell tower* - sprzedaż wieży (usuwa wieżę z mapy i zwraca część wydanego złota).

Koszt ulepszenia i wartość sprzedaży są prezentowane bezpośrednio w panelu. Przy braku środków przycisk ulepszenia pozostaje nieaktywny. Panel informacji o zaznaczonej wieży obejmuje prezentację parametrów oraz akcje *Upgrade* i *Sell* (Rys. 2.11).



Rysunek 2.11. Zrzut ekranu: panel informacji o zaznaczonej wieży oraz akcje *Upgrade* i *Sell*.

2.5.9. Uruchamianie fali i przeciwnicy

Kolejna fala przeciwników jest uruchamiana przyciskiem *Next wave*. Po rozpoczęciu fali przeciwnicy pojawiają się przy polu startowym i poruszają się wzdłuż drogi w kierunku pola docelowego. Dotarcie przeciwnika do celu zmniejsza wartość *HP*. Celem gracza jest eliminacja przeciwników za pomocą wież przed utratą wszystkich punktów życia. Przykład rozpoczętej fali i poruszających się przeciwników przedstawiono na rysunku 2.12.



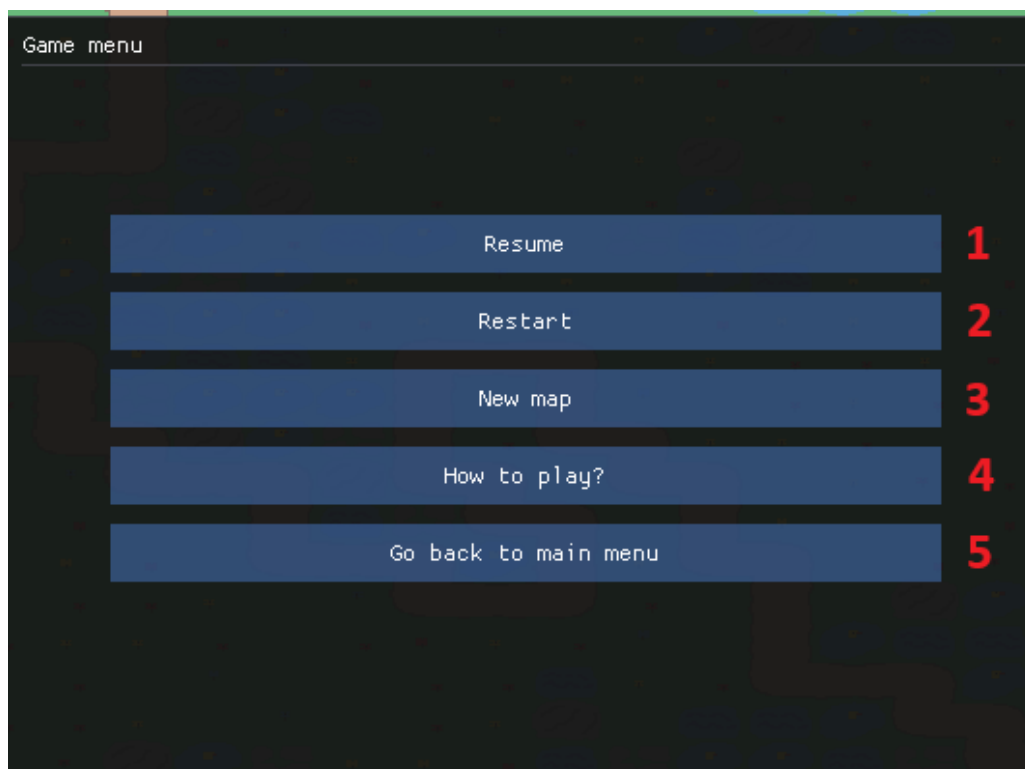
Rysunek 2.12. Zrzut ekranu: rozpoczęta fala - przeciwnicy poruszający się po drodze.

2.5.10. Menu pauzy

Menu pauzy jest dostępne po wybraniu przycisku *Menu* w panelu sterowania. W menu dostępne są opcje:

- *Resume* - powrót do rozgrywki,
- *Restart* - restart bieżącej mapy,
- *New map* - wygenerowanie nowej mapy,
- *How to play?* - wyświetlenie instrukcji,
- *Go back to main menu* - powrót do menu głównego.

Menu pauzy udostępnia wymienione opcje sterowania rozgrywką (Rys. 2.13).



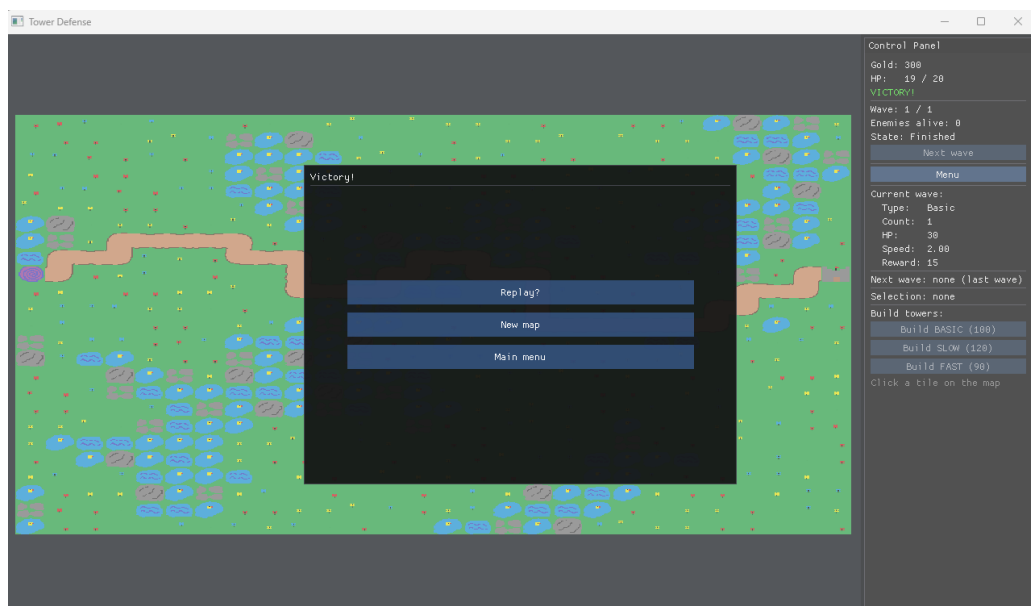
Rysunek 2.13. Zrzut ekranu: menu pauzy dostępne w trakcie rozgrywki.

2.5.11. Zakończenie gry

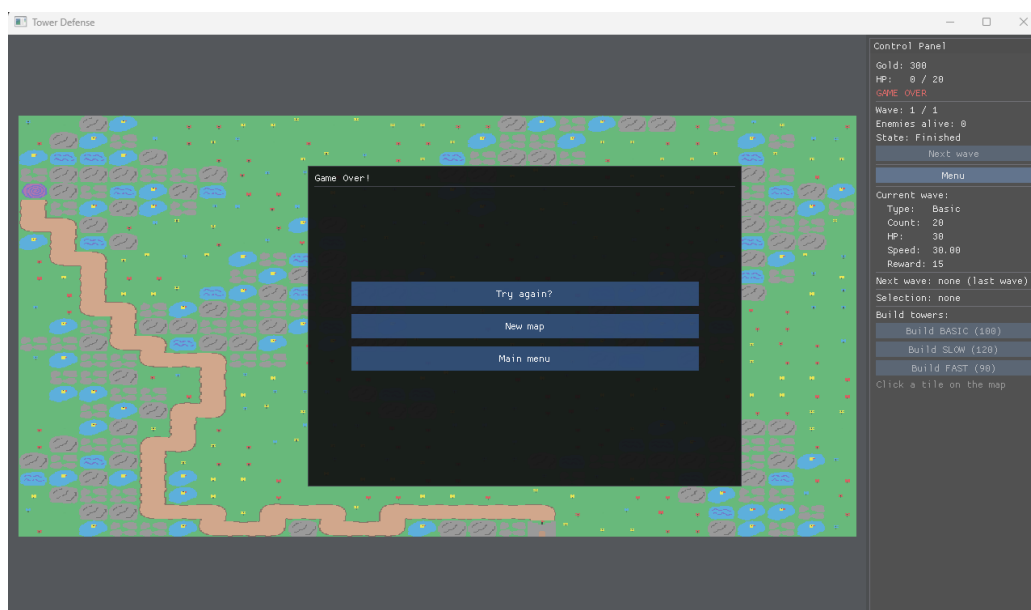
Gra kończy się:

- zwycięstwem po ukończeniu ostatniej fali (*Victory!*),
- porażką po spadku punktów życia do zera (*Game Over!*).

W obu przypadkach wyświetlane jest okno zakończenia, które umożliwia ponowne rozpoczęcie rozgrywki, wygenerowanie nowej mapy lub powrót do menu głównego (Rys. 2.14 i Rys. 2.15).



Rysunek 2.14. Zrzut ekranu: ekran zwycięstwa (*Victory!*).



Rysunek 2.15. Zrzut ekranu: ekran porażki (*Game Over!*).

Rozdział 3

Podsumowanie

Celem pracy było opracowanie i implementacja gry typu *Tower Defense* jako aplikacji desktopowej uruchamianej w środowisku Windows, z wykorzystaniem biblioteki OpenGL w profilu *Core* (wersja 3.3). W ramach realizacji projektu przygotowano kompletne środowisko rozgrywki obejmujące generowanie proceduralne mapy, system fal przeciwników, mechanikę stawiania wież, symulację pocisków oraz interfejs użytkownika.

Zaimplementowane mechanizmy umożliwiają przeprowadzenie pełnej rozgrywki złożonej z dziesięciu kolejnych fal. Gracz ma możliwość uruchamiania fal w dogodnym momencie, zarządzania ekonomią (złoto) oraz podejmowania decyzji strategicznych dotyczących doboru typów wież. Dodatkowo udostępniono funkcje ulepszania i sprzedaży wież, co pozwala na elastyczne dostosowywanie obrony do aktualnej sytuacji na planszy. Interfejs użytkownika zapewnia dostęp do najważniejszych informacji (stan zasobów, liczba punktów życia, postęp fal) i funkcji sterowania, w tym menu pauzy, restartu oraz generowania nowej mapy.

Najbardziej wymagającą częścią realizacji projektu było zaprojektowanie procedury generowania mapy, która zapewnia przechodnią i czytelną ścieżkę między punktem startowym i docelowym oraz jednocześnie pozwala na rozmieszczenie przeszkód w sposób nieblokujący trasy. Istotnym elementem było również zapewnienie spójności pomiędzy logiką gry a renderowaniem instancjonowanym, w szczególności w zakresie przekazywania danych o typach pól do shaderów.

Generowanie mapy zrealizowano dwuetapowo: w pierwszym kroku konstruowana jest przechodnia trasa przeciwników pomiędzy punktem startowym i docelowym, natomiast w drugim kroku pozostałe pola są wypełniane przeszkodami na podstawie wartości dwuwymiarowego szumu Perlina, z zachowaniem nienaruszalności wygenerowanej drogi.

Na podstawie wykonanych testów manualnych potwierdzono poprawność działania kluczowych funkcji aplikacji, w tym przejść pomiędzy stanami gry, uruchamiania fal, mechaniki wież oraz warunków zakończenia rozgrywki.

3.1. Wnioski

Na podstawie realizacji projektu oraz przeprowadzonych testów manualnych stwierdzono, że założone cele funkcjonalne zostały osiągnięte, a kluczowe mechanizmy rozgrywki działają poprawnie w typowych scenariuszach użytkowania. Zastosowanie instancjonowania w renderowaniu siatki ogranicza liczbę wywołań rysowania, natomiast generowanie proceduralne zapewnia zróżnicowanie rozgrywki przy zachowaniu przechodniej ścieżki. Ograniczenia bieżącej wersji (m.in. brak zapisu stanu gry i systemu dźwięku) wyznaczają naturalne kierunki dalszej rozbudowy.

3.2. Możliwości rozbudowy

W kolejnych etapach rozwoju aplikacji możliwe jest rozszerzenie projektu o dodatkowe elementy zwiększające atrakcyjność rozgrywki oraz kompletność produktu. Przykładowe kierunki rozbudowy obejmują:

- dodanie systemu dźwięku (efekty i muzyka tła) oraz konfiguracji głośności,
- zapis i odczyt stanu gry, co umożliwi kontynuowanie rozgrywki,
- wprowadzenie poziomów trudności (np. modyfikacja parametrów fal i ekonomii),
- rozbudowę typów przeciwników i wież (np. odporności, efekty obszarowe, wsparcie),
- ulepszenie generowania mapy o więcej wariantów biomu i czytelniejsze zasady rozmieszczania przeszkód,
- przygotowanie dodatkowych testów automatycznych dla wybranych komponentów logiki.

Spis rysunków

2.1.	Diagram klas przedstawiający kluczowe elementy architektury gry.	16
2.2.	Diagram przypadków użycia gry.	17
2.3.	Diagram aktywności: ulepszanie i sprzedaż wieży.	18
2.4.	Zrzut ekranu: menu główne gry.	39
2.5.	Zrzut ekranu: okno <i>How to play?</i> dostępne z menu głównego.	39
2.6.	Zrzut ekranu: widok rozgrywki po rozpoczęciu gry.	40
2.7.	Zrzut ekranu: panel sterowania bez zaznaczenia kafelka i wieży.	41
2.8.	Zrzut ekranu: zaznaczenie kafelka na mapie.	42
2.9.	Zrzut ekranu: panel budowy wież po zaznaczeniu kafelka możliwego do zabudowy.	43
2.10.	Zrzut ekranu: zbudowana wieża i komunikat blokujący ponowną budowę na tym samym kafelku.	44
2.11.	Zrzut ekranu: panel informacji o zaznaczonej wieży oraz akcje <i>Upgrade</i> i <i>Sell</i>	45
2.12.	Zrzut ekranu: rozpoczęta fala - przeciwnicy poruszający się po drodze.	46
2.13.	Zrzut ekranu: menu pauzy dostępne w trakcie rozgrywki.	47
2.14.	Zrzut ekranu: ekran zwycięstwa (<i>Victory!</i>).	48
2.15.	Zrzut ekranu: ekran porażki (<i>Game Over!</i>).	48

Spis tabel

2.1.	Scenariusz użycia: uruchomienie następnej fali.	19
2.2.	Scenariusz użycia: ulepszenie lub sprzedaż zaznaczonej wieży.	20
2.3.	Przykładowe testy manualne kluczowych funkcji aplikacji.	21
2.4.	Parametry bazowe wież dostępnych w grze.	29
2.5.	Parametry bazowe przeciwników występujących w grze.	29

Listings

2.1	Etap 1: wyznaczanie trasy (start/meta, próby generacji, reguły poprawności trasy) w <code>Grid::generateProceduralLevel</code> (fragment <code>Grid.cpp</code>).	22
2.2	Funkcje pomocnicze do obliczania szumu Perlina 2D (fragment <code>Grid.cpp</code>).	26
2.3	Etap 2: rozmieszczenie przeszkód na podstawie wartości szumu Perlina (fragment <code>Grid.cpp</code>).	28
2.4	Shader wierzchołków siatki (GLSL)	30
2.5	Fragment renderowania siatki z użyciem instancjonowania	30
2.6	Shader fragmentów siatki (GLSL)	31
2.7	Klasa <code>Shader</code> - plik nagłówkowy	36
2.8	Klasa <code>Shader</code> - implementacja	36

Bibliografia

- [1] S. Bubeck et al. *Search-based Procedural Content Generation: A Survey and Taxonomy*. 2023. URL: <https://arxiv.org/abs/2303.02293> (visited on 01/05/2026).
- [2] Mark Segal and Kurt Akeley, eds. *The OpenGL Graphics System: A Specification (Version 3.3 — Core Profile)*. Khronos OpenGL ARB. Mar. 11, 2010. URL: <https://registry.khronos.org/OpenGL/specs/gl/glspec33.core.pdf> (visited on 01/05/2026).
- [3] GLFW contributors. *GLFW — Documentation*. URL: <https://www.glfw.org/documentation.html> (visited on 01/05/2026).
- [4] GLAD contributors. *GLAD: Multi-Language GL/GLES/EGL/GLX/WGL Loader-Generator*. URL: <https://glad.dav1d.de/> (visited on 01/05/2026).
- [5] Omar Cornut. *Dear ImGui*. URL: <https://github.com/ocornut/imgui> (visited on 01/05/2026).
- [6] Sean Barrett. *stb_image.h — public domain image loader (v2.28)*. Plik nagłówkowy włączony do kodu źródłowego projektu (kopiowany wraz z projektem). Cytaty i opis pochodzą z komentarzy w pliku `stb_image.h` (sekcja nagłówkowa i Contributors). July 11, 2021.
- [7] Joey de Vries. *In Practice: 2D Game — Breakout*. URL: <https://learnopengl.com/In-Practice/2D-Game/Breakout> (visited on 01/05/2026).
- [8] *Taxicab geometry*. URL: https://en.wikipedia.org/wiki/Taxicab_geometry (visited on 01/05/2026).
- [9] *Taxicab Metric*. URL: <https://mathworld.wolfram.com/TaxicabMetric.html> (visited on 01/05/2026).
- [10] Ken Perlin. *Chapter 26: Implementing Improved Perlin Noise*. URL: <https://developer.nvidia.com/gpugems/gpugems2/part-iii-high-quality-rendering/chapter-26-implementing-improved-perlin-noise> (visited on 01/05/2026).